14.04.2026

# NP- алгоритмы.

***Филиппов Михаил Витальевич***

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

Новосибирский государственный университет

*НАСТОЯЩАЯ НАУКА

# Давайте познакомимся

## Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.

Новосибирский государственный университет
*НАСТОЯЩАЯ НАУКА

# Адженда

**NP-алгоритмы**

**90 минут**

# Введение

Как и раньше: ***критерий эффективности*** – полиномиальное время выполнения. У такого конкретного определения есть одно важное преимущество: оно позволяет привести математическое доказательство того, что *некоторые задачи не могут быть решены при помощи алгоритмов с полиномиальным временем* (то есть «эффективных»).Алгоритм с полиномиальным временем для некоторых задач неизвестны, но и доказательств того, что таких алгоритмов не существует, пока нет.

***Три класса проблем разрешимости***:

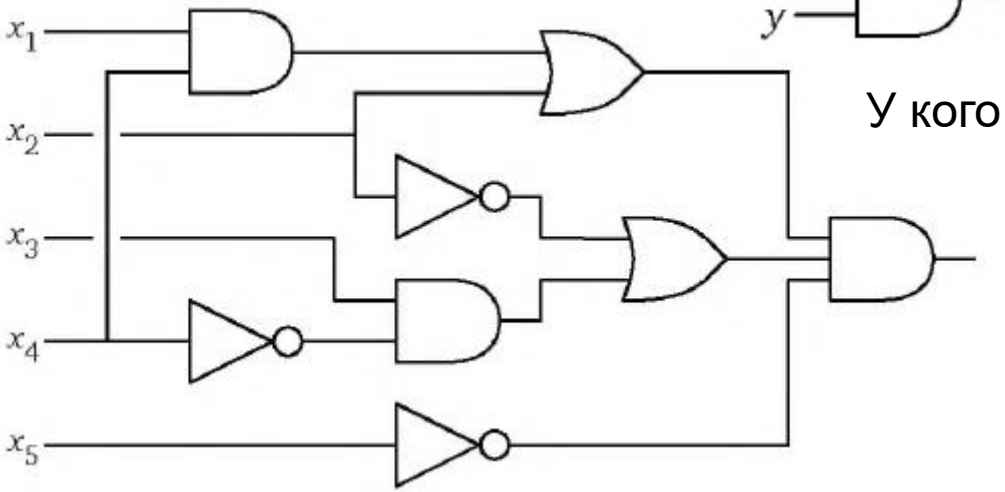
- P - множество проблем разрешимости, которые можно решить за полиномиальное время.
- NP - множество проблем разрешимости со следующим свойством: если ответ Yes, то существует доказательство этого факта, которое можно проверить за полиномиальное время
- co-NP - по существу, противоположно NP. Если ответом на задачу из множества co-NP является No, то существует доказательство этого факта, которое можно проверить за полиномиальное время.

**Определение 27.1.** ***NP-трудные задачи*** – задачи, в которых если существует алгоритм с полиномиальным временем выполнения – существует алгоритм с полиномиальным временем выполнения для каждой задачи в NP.

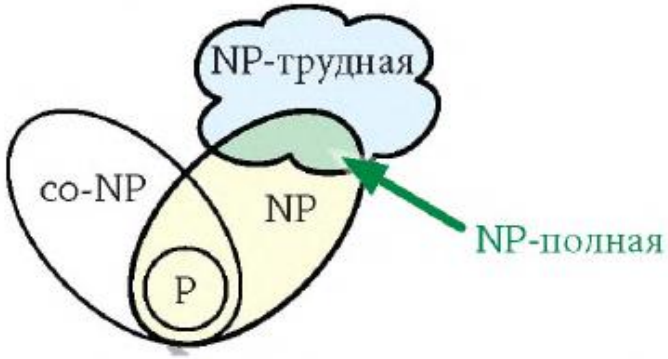
**Определение 27.2.** ***NP-полные задачи*** – это набор задач, для одной из которых найден алгоритм с полиномиальным временем выполнения, что означает существование алгоритма для всех таких задач.

# Полиномиальное сведение

Какая математической характеристика такого большого класса?
«задача X обладает как минимум не меньшей сложностью, чем задача Y».
Докажем, что «черный ящик», способный решить задачу X, также сможет решить задачу Y.
Предположим, что задача X в нашей вычислительной модели может быть напрямую решена с полиномиальным временем.
***Утверждение 27.1.*** Допустим, $Y \leq_P X$. Если задача X решается за полиномиальное время, то и задача Y может быть решена за полиномиальное время.
***Утверждение 27.2.*** Допустим, $Y \leq_P X$. Если задача Y не решается за полиномиальное время, то и задача X не может быть решена за полиномиальное время.



У кого лампочка загорится, тому много денег)

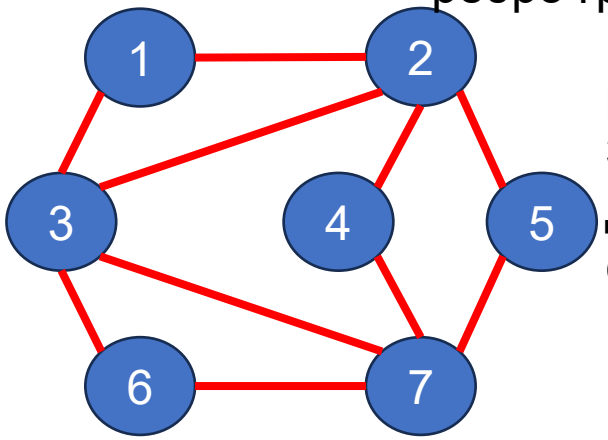


# Независимое множество и вершинное покрытие

Алгоритм с полиномиальным *временем для нее неизвестен*, но мы также не знаем, как доказать, что такого алгоритма не существует.

***Формулировка задачи о независимом множестве:*** в графе G = (V, E) множество узлов S ⊆ V называется независимым, если никакие два узла в S не соединены ребром. Найти малое независимое множество в графе несложно (например, одиночный узел образует независимое множество); трудности возникают при поиске больших независимых множество так как требуется построить большой набор узлов, в котором нет соседних узлов. Например, множество узлов {3, 4, 5} является независимым множеством размера 3 в графе на рис. 1.1, тогда как множество узлов {1, 4, 5, 6} является независимым множеством большего размера.

Вершинное покрытие графа — это подмножество вершин графа, такое, что каждое ребро графа инцидентно (соединено) хотя бы одной вершине из этого подмножества.




Граф, для которого размер наибольшего независимого множества равен 4, а наименьшее вершинное покрытие имеет размер 3

Задача о независимом множестве будет сформулирована следующим образом:

**Для заданного графа G и числа k содержит ли G независимое множество с размером не менее k?**

Мы не знаем, как решить задачу о независимом множестве или задачу о вершинном покрытии за полиномиальное время; но что можно сказать об их относительной сложности?

# Сведение задач о независимом множестве и вершинном покрытии

***Утверждение 27.3.*** Имеется граф G = (V, E). S является независимым множеством в том и только в том случае, если его дополнение V−S является вершинным покрытием.

Доказательство. Предположим, что S является независимым множеством. Рассмотрим произвольное ребро e = (u, v). Из независимости S следует, что u и v не могут одновременно принадлежать S; следовательно, один из этих концов должен принадлежать V−S. Отсюда следует, что у каждого ребра как минимум один конец принадлежит V−S, а значит, V−S является вершинным покрытием.

И наоборот, предположим, что V−S является вершинным покрытием. Рассмотрим два любых узла u и v в S. Если бы они были соединены ребром e, то ни один из концов e не принадлежал бы V−S, что противоречило бы предположению о том, что V−S является вершинным покрытием. Отсюда следует, что никакие два узла в S не соединены ребром, а значит, S является независимым множеством. ■

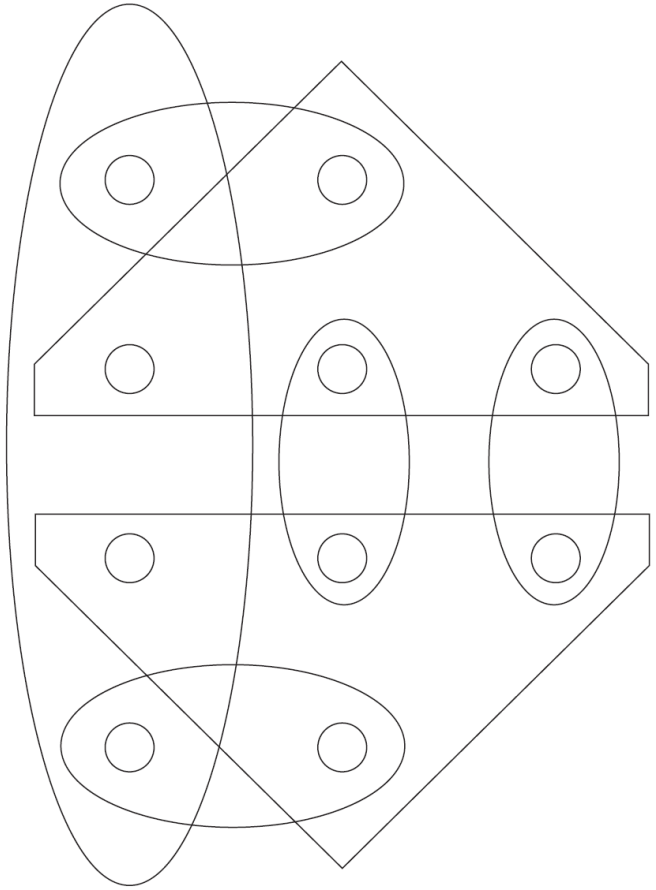
***Утверждение 27.4.*** Независимое множество $\leq_P$ Вершинное покрытие.

Доказательство. Имея «черный ящик» для решения задачи о вершинном покрытии, мы могли бы решить, содержит ли G независимое множество с размером не менее k, обратившись к «черному ящику» с вопросом о том, содержит ли G вершинное покрытие с размером не более n − k. ■

***Утверждение 27.5.*** Вершинное покрытие $\leq_P$ Независимое множество.

Доказательство. Имея «черный ящик» для решения задачи о независимом множестве, мы могли бы решить, содержит ли G вершинное покрытие размером не более k, обратившись к «черному ящику» с вопросом о содержит ли G независимое множество с размером не менее n − k. ■

# Сведение к более общему случаю: вершинное покрытие к покрытию множества



Задача о независимом множестве и задача о вершинном покрытии представляют собой ***две разные категории задач***.

- Задача о независимом множестве может рассматриваться как «задача упаковки»: требуется «упаковать» как можно больше вершин с учетом конфликтов (ребер), препятствующих нам в этом.
- Задача о вершинном покрытии может рассматриваться как «задача покрытия»: требуется экономно «накрыть» все ребра в графе с использованием минимального количества вершин.

Задано множеств U из n элементов, набор $S_1$, ..., $S_m$ подмножеств U и число k. Существует ли набор из не более чем k таких множеств, объединение которых равно всему множеству U?

Представьте, например, что имеются m программных компонентов и множество U из n функций, которыми должна обладать система. i-й компонент реализует множество $S_i \subseteq U$ функций. В задаче покрытия множества требуется включить небольшое количество компонентов, чтобы система поддерживала все n функций.

**Рисунок.** Пример задачи покрытия множества

# Сведение к более общему случаю

***Утверждение 27.6.*** Вершинное покрытие $\leq_P$ Покрытие множества.

**Доказательство.** Допустим, имеется «черный ящик», который умеет решать задачу о покрытии множества, и произвольный экземпляр задачи о вершинном покрытии, заданный графом G = (V, E) и числом k. *Как использовать «черный ящик»* для решения этой задачи?

***Цель*** — найти покрытие для ребер E, поэтому мы формулируем экземпляр задачи покрытия множества, в которой универсальное множество U равно E. Каждый раз, когда мы выбираем вершину в задаче вершинного покрытия, мы покрываем все ребра, инцидентные этой вершине; следовательно, для каждой вершины $i \in V$ в экземпляр задачи покрытия множества добавляется множество $S_i \subseteq U$, состоящее из всех ребер G, инцидентных i.

*Утверждается,* что U может быть покрыто с использованием не более k из множеств $S_1, \ldots, S_n$ в том, и только в том случае, если G имеет вершинное покрытие с размером не более k. Это утверждение доказывается очень просто. Если $l \leq k$ множеств $S_{i_1}, \ldots, S_{i_l}$ покрывают U, то каждое ребро в G инцидентно одной из вершин $i_1 \ldots i_l$, а значит, множество $\{i_1 \ldots i_l\}$ является вершинным покрытием G с размером $l \leq k$. И наоборот, если $\{i_1 \ldots i_l\}$ является вершинным покрытием G с размером $l \leq k$, то множества покрывают U.

Итак, для заданного экземпляра задачи о вершинном покрытии формулируется экземпляр задачи покрытия множества, описанный выше, который передается «черному ящику». Положительный ответ на исходный вопрос дается в том, и только в том случае, если «черный ящик» отвечает положительно.■

***Утверждение 27.7.*** Независимое множество $\leq_P$ Упаковка множества.

# Задача SAT

Допустим, имеется множество X из n булевых переменных $x_1$, ..., $x_n$; каждая переменная может принимать значение 0 или 1 (эквиваленты false и true). **Определение 27.3.** ***Литералом по X*** называется одна из переменных $x_i$ или ее отрицание $\overline{x_i}$. Наконец, условием называется обычная дизъюнкция литералов

$$t_1 \vee t_2 \vee \cdots \vee t_l.$$

(Еще раз: все $t_i \in \{x_1, x_2, \dots, x_n, \overline{x_1}, \dots, \overline{x_n}\}$). Мы говорим, что условие имеет длину l, если оно содержит l литералов.

**Определение 27.4.** ***Логическим присваиванием*** для X называется присваивание значения 0 или 1 каждому $x_i$; другими словами, это функция v : X → {0,1}. Присваивание v неявно задает $\overline{x_i}$ значение истинности, противоположное $x_i$. Присваивание выполняет условие C, если после него C дает результат 1 по условиям булевой логики; это эквивалентно требованию о том, чтобы по крайней мере один из литералов в C имел значение 1. Присваивание выполняет совокупность условий $C_1$, ..., $C_k$, если в результате его все $C_i$ дают результат 1; иначе говоря, если в результате его конъюнкция дает результат 1:

$$C_1 \wedge C_2 \wedge \cdots \wedge C_k.$$

**Определение 27.5.** В этом случае ∧ называется выполняющим присваиванием в отношении $C_1$, ..., $C_k$, а набор условий $C_1$, ..., $C_k$ называется выполнимым.

***Формулировка задачи выполнимости, также обозначаемой SAT:***

Для заданного множества условий $C_1$, ..., $C_k$ по множеству переменных X = {$x_1$, ..., $x_n$} существует ли выполняющее логическое присваивание?

# SAT

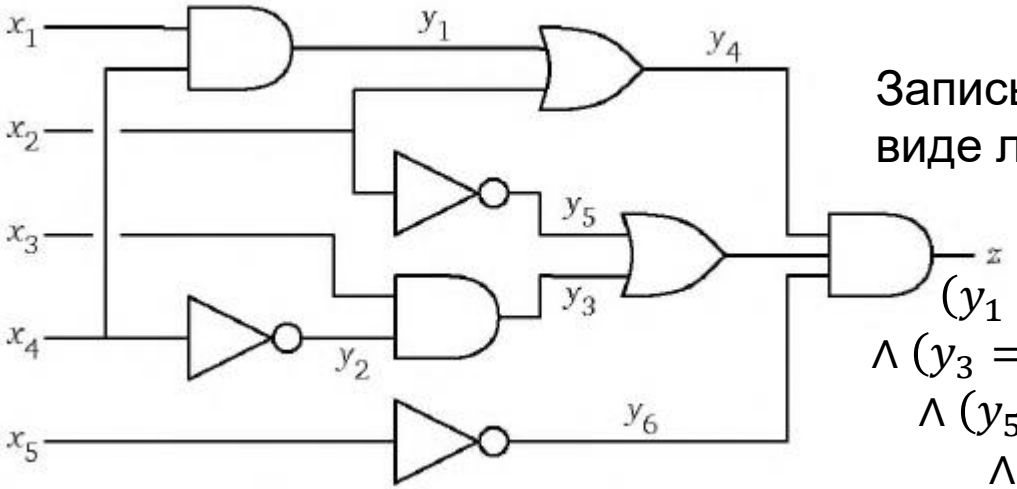
- Абстрактная задача
- Формулируются в булевой записи.
- Применяется для моделирования широкого спектра задач, в которых требуется присваивать значения условных переменных для выполнения заданного набора ограничений: например, подобные формальные конструкции часто встречаются в области искусственного интеллекта.

***Вопрос:*** возможно ли присвоить логические значения переменным a, b, c, … так, чтобы вычисление всей формулы дало результат True.

**Пример.** Допустим, имеются три условия:

$$(x_1 \vee \overline{x_2}), (\overline{x_1} \vee \overline{x_3}), (x_2 \vee \overline{x_3}).$$

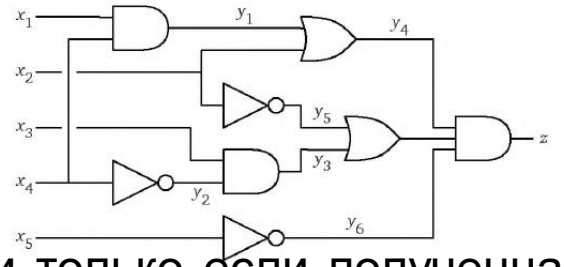
Что является выполняющим присваиванием?




Запись логической схемы в виде логической формулы

$$(y_1 = x_1 \wedge x_4) \wedge (y_2 = \overline{x_4}) \wedge$$
$$\wedge (y_3 = x_3 \wedge y_2) \wedge (y_4 = y_1 \vee x_2) \wedge$$
$$\wedge (y_5 = \overline{x_2}) \wedge (y_7 = y_3 \vee y_5) \wedge$$
$$\wedge (z = y_4 \wedge y_7 \wedge y_6) \wedge z$$

# Задача SAT




***Утверждение 27.7.*** Исходная схема К выполнима, если и только если полученная в итоге формула Ф выполнима.

**=>.** Если задано множество входных данных, выполняющих логическую схему К, то мы можем вывести выполнимое присваивание для формулы Ф, вычислив выход каждого логического вентиля в схеме К.

**<=** Если задано выполнимое присваивание для формулы Ф, то мы можем получить выполнимые входные данные для логической схемы, просто игнорируя переменные внутренних проводов y и выходную переменную z.

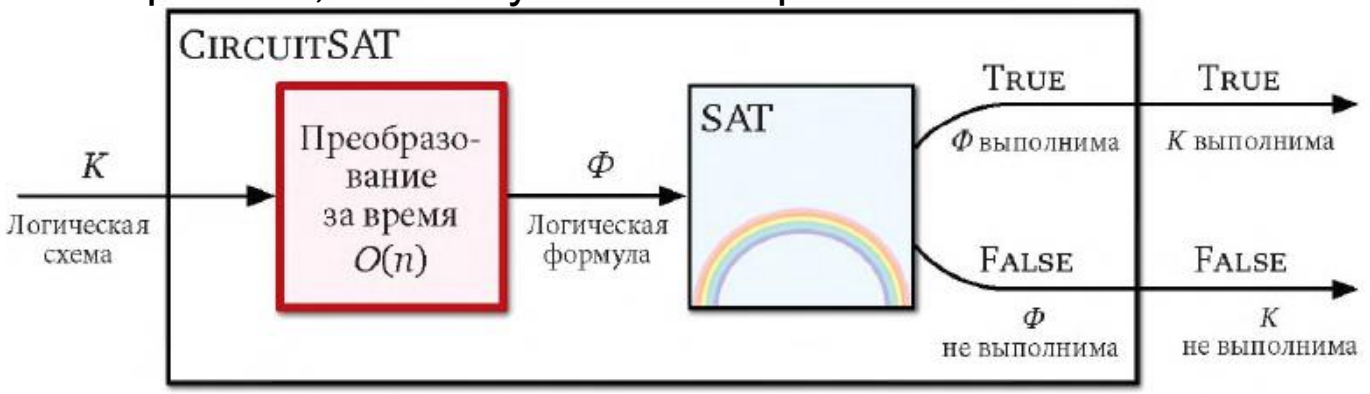
Все преобразование в целом от логической схемы к логической формуле можно выполнить ***за линейное время***. Кроме того, размер полученной формулы больше любого разумного представления логической схемы всего лишь с постоянным коэффициентом.

Предположим, чисто теоретически, что существует алгоритм, который может определить за полиномиальное время, является ли заданная логическая формула выполнимой.

Преобразование К в Ф требует всего лишь линейного времени, поэтому весь алгоритм CircuitSat также выполняется за полиномиальное время.

$$Termites(n) < O(n) + T(O(n)).$$

***Вывод:*** любой алгоритм с полиномиальным временем выполнения для Sat должен давать нам алгоритм с полиномиальным временем выполнения для CircuitSat, что, в свою очередь, предполагает **P = NP**. Поэтому Sat является NP-трудной задачей.



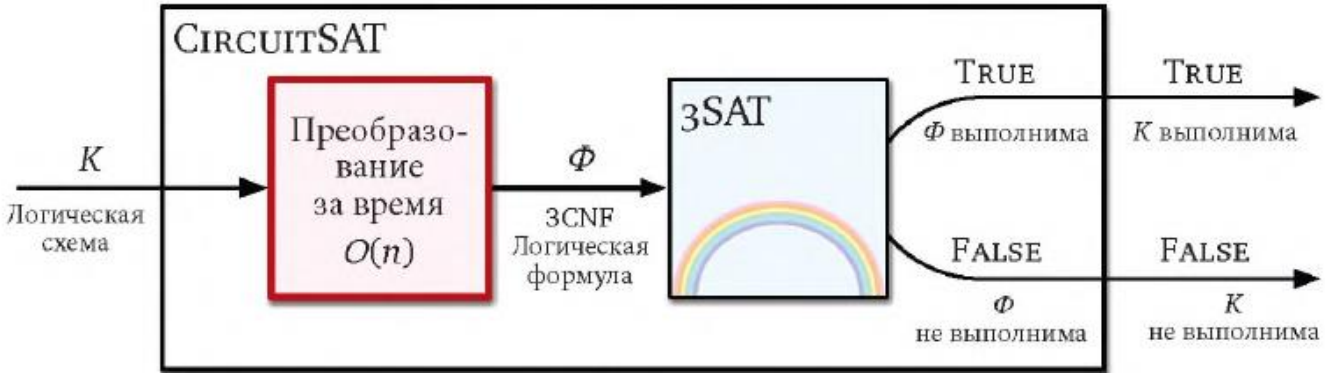
# Задача 3-SAT

**Определение 27.6.** Логическая формула находится в конъюнктивной нормальной форме(КНФ) (conjunctive normal form - CNF), если представляет собой конъюнкцию (AND) нескольких компонент (clauses), каждая из которых является дизъюнкцией (OR) нескольких литералов (literals), каждый из которых является переменной или ее отрицанием. Например:

$$\overbrace{(a \vee b \vee c \vee d)}^{\text{компонента}} \wedge (b \vee \bar{c} \vee \bar{d}) \wedge (a \vee c \vee d) \wedge (a \vee \bar{b}).$$

Формула выполнима, если можно расставить переменным значения Истина / Ложь так, чтобы вся формула стала истинной. А чтобы вся КНФ стала истинной — каждая клауза должна быть истинной. А чтобы клауза была истинной — хотя бы одно выражение внутри неё должно быть истинным.

Существует частный случай SAT, обладающий эквивалентной сложностью, но при этом более понятный; в нем все условия содержат ровно три литерала (соответствующих разным переменным). Назовем эту задачу задачей 3CNF-SAT, или 3-SAT. Он очень полезен при решении других задач.




Для заданного множества условий $C_1, \ldots, C_k$, каждое из которых имеет длину 3, по множеству переменных $X = \{x_1, \ldots, x_n\}$, существует ли выполняющее логическое присваивание?

# Задача 3-SAT

Если задана произвольная логическая схема К, то мы выполняем преобразование К в равнозначную формулу 3CNF в несколько этапов. Эта редукция, за исключением самого последнего этапа, в действительности была описана Григорием Цейтиным в 1966 г. Будем описывать каждый этап, одновременно доказывая, что он корректен.

- Убедимся, что каждый вентиль AND и OR в схеме К имеет ровно два входа. Если какой-либо вентиль имеет k > 2 входов, то заменим его на двоичное дерево из k - 1 двоичных вентилей. Назовем полученную схему К'. Схемы К и К' логически равнозначны.
- Перепишем схему К' в виде логической формулы $\Phi_1$, с одной компонентой для каждого вентиля точно так же, как в предыдущей редукции к Sat.
- Заменим каждую компоненту в $\Phi_1$ на формулу CNF. Существует только три типа компонент в $\Phi_1$ - по одной для каждого типа вентилей в схеме К':

$$a = b \wedge c \rightarrow \left(\bar{a} \vee \bar{b} \vee \bar{c}\right) \wedge (\bar{a} \vee b) \wedge (\bar{a} \vee c)$$

$$a = b \vee c \rightarrow (\bar{a} \vee b \vee c) \wedge \left(a \vee \bar{b}\right) \wedge (a \vee \bar{c})$$

$$a = b \rightarrow (a \vee b) \wedge (\bar{a} \vee \bar{b}).$$

Назовем полученную формулу CNF $\Phi_2$. Поскольку $\Phi_1$ и $\Phi_2$ логически равнозначны, каждое выполнимое присваивание для Ф также является выполнимым присваиванием для $\Phi_2$ и наоборот.

# Задача 3-SAT

- Заменим каждую компоненту в $\Phi_2$ на формулу 3CNF. Каждая компонента в $\Phi_2$ содержит не более трех литералов, но нам нужны компоненты с ровно тремя литералами. Для получения формулы 3CNF мы разворачиваем каждую компоненту из двух литералов в $\Phi_2$ в две компоненты из трех литералов, вводя одну новую переменную, а последнюю компоненту из одного литерала в $\Phi_2$ - в четыре компоненты из трех литералов, вводя две новые переменные.

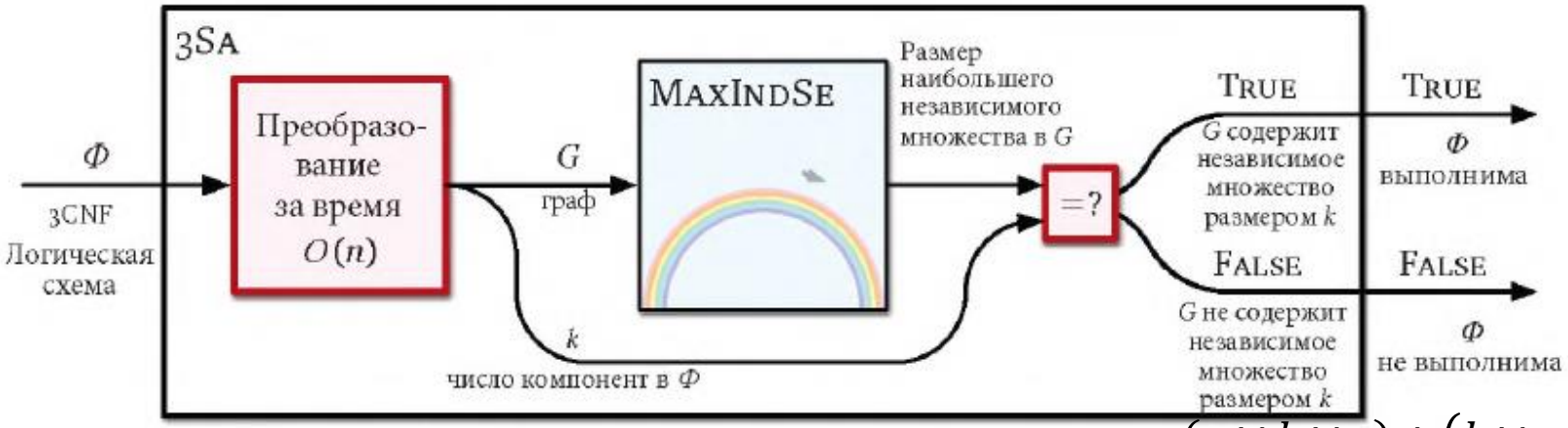
$$a \vee b \rightarrow (a \vee b \vee x) \wedge (a \vee b \vee \bar{x})$$

$$z \rightarrow (z \vee x \vee y) \wedge (z \vee \bar{x} \vee y) \wedge (z \vee x \vee \bar{y}) \wedge (z \vee \bar{x} \vee \bar{y}).$$

Назовем полученную формулу 3CNF $\Phi_3$ . Каждое выполнимое присваивание для Ф., можно преобразовать в выполнимое присваивание для $\Phi_2$, присваивая произвольные значения новым переменным (x и y). И обратно: каждое выполнимое присваивание для $\Phi_3$ можно преобразовать в выполнимое присваивание для $\Phi_2$, игнорируя новые переменные.

$$(y_1 = x_1 \wedge x_4) \wedge (y_2 = \overline{x_4}) \wedge (y_3 = x_3 \wedge y_2) \wedge (y_4 = y_1 \vee x_2) \wedge (y_5 = \overline{x_2}) \wedge (y_7 = y_3 \vee y_5) \wedge (z = y_4 \wedge y_7 \wedge y_6) \wedge z$$
$$\leftrightarrow (y_1 \vee \overline{x_1} \vee \overline{x_4}) \wedge (\overline{y_1} \vee x_1 \vee z_1) \wedge (\overline{y_1} \vee x_1 \vee \overline{z_1}) \wedge (\overline{y_1} \vee x_4 \vee z_2) \wedge (\overline{y_1} \vee x_4 \vee \overline{z_2}) \wedge (y_2 \vee x_4 \vee z_3) \wedge (y_2 \vee x_4 \vee \overline{z_3})$$
$$\wedge (\overline{y_2} \vee \overline{x_4} \vee z_4) \wedge (\overline{y_2} \vee \overline{x_4} \vee \overline{z_4}) \wedge (y_3 \vee \overline{x_3} \vee \overline{y_2}) \wedge (\overline{y_3} \vee x_3 \vee z_5) \wedge (\overline{y_3} \vee x_3 \vee \overline{z_5}) \wedge (\overline{y_3} \vee y_2 \vee z_6) \wedge (\overline{y_3} \vee y_2 \vee \overline{z_6})$$
$$\wedge (\overline{y_4} \vee y_1 \vee x_2) \wedge (y_4 \vee \overline{x_2} \vee z_7) \wedge (y_4 \vee \overline{x_2} \vee \overline{z_7}) \wedge (y_4 \vee \overline{y_1} \vee z_8) \wedge (y_4 \vee \overline{y_1} \vee \overline{z_8}) \wedge (y_5 \vee x_2 \vee z_9) \wedge (y_5 \vee x_2 \vee \overline{z_9})$$
$$\wedge (\overline{y_5} \vee \overline{x_2} \vee \overline{z_{10}}) \wedge (\overline{y_5} \vee \overline{x_2} \vee z_{10}) \wedge (y_6 \vee x_5 \vee z_{11}) \wedge (y_6 \vee x_5 \vee \overline{z_{11}}) \wedge (\overline{y_6} \vee \overline{x_5} \vee z_{12}) \wedge (\overline{y_6} \vee \overline{x_5} \vee \overline{z_{12}}) \wedge (\overline{y_7} \vee y_3$$
$$\vee y_5) \wedge (y_7 \vee \overline{y_3} \vee z_{13}) \wedge (y_7 \vee \overline{y_3} \vee \overline{z_{13}}) \wedge (y_7 \vee \overline{y_5} \vee z_{14}) \wedge y_7 \vee \overline{y_5} \vee \overline{z_{14}}) \wedge (y_8 \vee \overline{y_4} \vee \overline{y_7}) \wedge (\overline{y_8} \vee y_4 \vee z_{15}) \wedge (\overline{y_8}$$
$$\vee y_4 \vee \overline{z_{15}}) \wedge (\overline{y_8} \vee y_7 \vee z_{16}) \wedge (\overline{y_8} \vee y_7 \vee \overline{z_{16}}) \wedge (y_9 \vee \overline{y_8} \vee \overline{y_6}) \wedge (\overline{y_9} \vee y_8 \vee z_{17}) \wedge (\overline{y_9} \vee y_6 \vee z_{18}) \wedge (\overline{y_9} \vee y_6 \vee \overline{z_{18}})$$
$$\wedge (\overline{y_9} \vee y_8 \vee \overline{z_{17}}) \wedge (y_9 \vee z_{19} \vee z_{20}) \wedge (y_9 \vee \overline{z_{19}} \vee z_{20}) \wedge (y_9 \vee z_{19} \vee \overline{z_{20}}) \wedge (y_9 \vee \overline{z_{19}} \vee \overline{z_{20}})$$

# Сведение задачи 3-SAT к задаче о независимом множестве



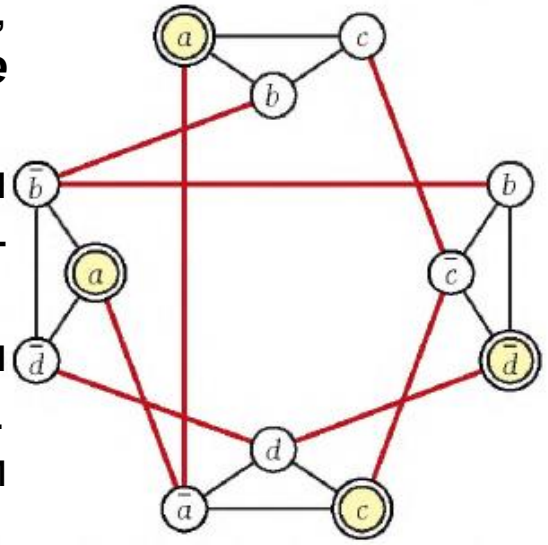

Утверждается, что исходный экземпляр задачи 3-SAT выполним в том и только в том случае, если построенный граф G имеет независимое множество с размером не менее k.

$$(a \vee b \vee c) \wedge (b \vee c \vee d) \wedge (\bar{a} \vee c \vee d) \wedge (a \vee \bar{b} \vee \bar{d})$$

Существует логическое присваивание v для переменных в экземпляре 3-SAT, обладающее тем свойством, что метки всех узлов S дают результат 1. ***Как же построить такое присваивание v?***

- Для каждой переменной $x_i$, если ни $x_i$, ни $\overline{x_i}$ не используется как метка узла в S, мы присваиваем $v(x_i) = 1$. В противном случае либо $x_i$, либо $\overline{x_i}$ является меткой узла - противоречие предположению о том, что S- независимое множество.
- Если $x_i$ - метка узла в S, мы присваиваем $v(x_i) = 1$; в противном случае выполняется присваивание $v(x_i) = 0$. При таком построении v все метки узлов в S дают результат 1.

Так как G содержит независимое множество с размером не менее k только в том случае, если исходный экземпляр 3-SAT был выполнимым, сведение завершено. ■



# Сведение задачи 3-SAT к задаче о независимом множестве

***Утверждение 27.9.*** 3-SAT $\leq_P$ Независимое множество.

<u>Доказательство.</u> Имеется «черный ящик» для задачи о независимом множестве; мы хотим решить экземпляр задачи 3-SAT, состоящий из переменных $X = \{x_1, ..., x_n\}$ и условий $C_1, ..., C_k$.

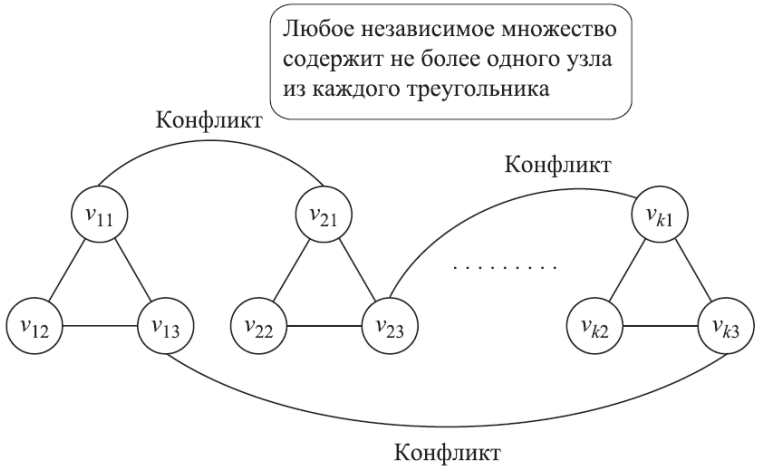
Две концептуально различающиеся точки зрения на экземпляр 3-SAT:

- Способ представления экземпляра 3-SAT - ∀ из n переменных принимается независимое решение 0/1, а успех - достижение одного из трех способов выполнения каждого условия.
- Успех достигается в том случае, если вы сможете выбрать литерал из каждого условия так, чтобы никакие два выбранных литерала не «конфликтовали»; (один равен $x_i$, а другой $\overline{x_i}$).

Следующее сведение основано *на втором* представлении 3-SAT. Строим граф G = (V, E), состоящий из 3k узлов, сгруппированных в k треугольников (для i = 1, 2, ..., k: $v_{i1}$, $v_{i2}$, $v_{i3}$, соединенные друг с другом ребрами). Каждой вершине присваивается метка; $v_{ij}$ помечается j-м литералом из условия $C_i$ экземпляра 3-SAT.

Две вершины не могут быть выбраны из одного треугольника, они состоят из всех способов выбора одной вершины из каждого треугольника.

Конфликты будут кодироваться добавлением новых ребер в граф: каждую пару вершин, метки которых соответствуют конфликтующим литералам, мы соединяем ребром. *Приведет ли это к уничтожению всех независимых множеств размера k?* Зависит от того, можно ли выбрать один узел из каждого треугольника, чтобы при этом не были выбраны конфликтующие пары узлов. Но ведь именно то, что требуется в экземпляре 3-SAT!



# Транзитивность сведения

Мы рассмотрели несколько разных сложных задач, относящихся к разным классам, и выяснили, что они тесно связаны друг с другом. Мы можем вычислить несколько дополнительных отношений, используя факт транзитивности $\leq_P$.

***Утверждение 27.10.*** Если $Z \leq_P Y$ и $Y \leq_P X$, то $Z \leq_P X$.

Доказательство. Располагая «черным ящиком» для решения X, мы покажем, как решить экземпляр задачи Z; по сути, мы просто выполняем композицию двух алгоритмов, подразумеваемых $Z \leq_P Y$ и $Y \leq_P X$. Алгоритм для Z выполняется с использованием «черного ящика» для Y; но каждое обращение к «черному ящику» для Y моделируется полиномиальным количеством шагов, использующих алгоритм, решающий экземпляры Y с использованием «черного ящика» для X. ■

Свойство транзитивности весьма полезно. Например, так как мы доказали, что 3-SAT $\leq_P$ Независимое множество $\leq_P$ Вершинное покрытие $\leq_P$ Покрытие множества, можно сделать вывод, что 3-SAT $\leq_P$ Покрытие множества.

# Эффективная сертификация и определение NP

Компоненты в изучении вычислительной неразрешимости

- сведение задач;
- получение характеристик класса задач, с которыми мы имеем дело.

Здесь важен контраст между поиском решения и проверкой предложенного решения.

Какие «доказательства» можно было бы предъявить, чтобы убедить вас за полиномиальное время в невыполнимости задачи?

***Формализация:***

- Входные данные вычислительной задачи могут быть закодированы в виде конечной двоичной строки s.
- Задача принятия решения X будет описываться множеством строк, для которых возвращается ответ «да».
- Алгоритм A для задачи принятия решения получает входную строку s и возвращает «да» или «нет» — это возвращаемое значение будет обозначаться A(s). Алгоритм A решает задачу X, если ∀s условие A(s) = да выполняется в том, и только в том случае, если $s \in X$.

A имеет полиномиальное время выполнения, если существует такая полиномиальная функция p(·), что ∀ s алгоритм A завершается для s не более чем за O(p(|s|)) шагов.

***Ищем:*** Множество P всех задач X, для которых существует алгоритм A с полиномиальным временем выполнения, решающий X.

# Эффективная сертификация

Как же формализовать идею о том, что решение задачи может быть проверено эффективно независимо от того, насколько эффективно может решаться сама задача?

Структура «алгоритма проверки» для задачи X отличается от структуры алгоритма, ищущего решение; для «проверки» решения необходима входная строка s и отдельная строка t («сертификат»), доказывающая, что s является «положительным» экземпляром X.

**Определение 27.5.** Алгоритм B называется эффективным сертифицирующим алгоритмом для задачи X, если он обладает следующими свойствами:

- B — алгоритм с полиномиальным временем, получающий два входных аргумента s и t;
- существует такая полиномиальная функция p, что для каждой строки s условие $s \in X$ выполняется в том, и только в том случае, если существует строка t, для которой $|t| \leq P(|s|)$ и B(s, t) = да.

Эффективный сертифицирующий алгоритм следует рассматривать как подход к задаче X с «управленческой» точки зрения.

Эффективный сертифицирующий алгоритм B может использоваться как центральный компонент алгоритма «грубой силы» для задачи X: для входной строки s проверить все строки t длины $\leq P(|s|)$ и проверить, выполняется ли условие B(s, t) = да для каких-либо из этих строк. Однако существование B не дает никакого очевидного способа построения эффективного алгоритма, который решает X.

# NP: класс задач

**Определим NP** как множество задач, для которых существует эффективный сертифицирующий алгоритм. Операция поиска строки t, с которой эффективный сертифицирующий алгоритм примет ввод s, часто рассматривается как недетерминированный поиск по пространству возможных доказательств t; по этой причине термин **NP** был выбран как сокращение для **«Nondeterministic Polynomial time»** («недетерминированное полиномиальное время»).

***Утверждение 27.11.*** $P \subseteq NP$.

**Доказательство.** Рассмотрим задачу $X \in P$; это означает, что существует алгоритм с полиномиальным временем, который решает X. Чтобы показать, что $X \in NP$, необходимо показать, что существует эффективный сертифицирующий алгоритм B для X.

Сделать это очень просто; B строится по следующей схеме.

- При получении входной пары (s, t) сертифицирующий алгоритм B просто возвращает значение A(s). (Считайте B своего рода «прагматиком», который игнорирует предложенное доказательство t и просто решает задачу своими силами.)

Почему B является эффективным сертификатором для X?

Очевидно, он выполняется с полиномиальным временем, как и A. Если строка $s \in X$, то для всех t длины не более $p(|s|)$ имеем B(s, t) = да. С другой стороны, если $s \notin X$, то для всех t длины не более $p(|s|)$ имеем B(s, t) = нет. ■

# NP: класс задач

**Как сертифицировать задачи.** Например:

- для задачи 3-SAT сертификат t является результатом присваивания булевых значений переменным; сертифицирующий алгоритм B проверяет заданное множество условий в соответствии с этим присваиванием;
- для задачи о независимом множестве сертификат t является описанием множества, содержащим не менее k вершин; сертифицирующий алгоритм B проверяет, что среди этих вершин нет двух, соединенных ребром;
- для задачи покрытия множества сертификат t представляет собой список k множеств из заданного набора; сертифицирующий алгоритм проверяет, что объединение этих множеств равно базовому множеству U.

Истинно ли равенство ***P = NP?*** Одна из самых известных задач в современной теории вычислений. Не достигнув прогресса в разрешении вопроса P = NP, ученые обратились к более доступному вопросу: какие задачи в NP обладают наибольшей сложностью?

Способ определения «наиболее сложной» задачи X основан на следующих двух свойствах:

**(i)** $X \in NP$ **(ii)** для всех $Y \in NP$ $Y \leq_P X$.

**Определение 27.7.** NP-полная задача X это такая задача,что каждая задача в NP сводится к X.

***Утверждение 27.12.*** Допустим, X является NP-полной задачей. Тогда X решается за полиномиальное время в том, и только в том случае, если P = NP.

Доказательство. Очевидно, если P = NP, то задача X решается за полиномиальное время, так как она принадлежит NP. И наоборот, предположим, что X решается за полиномиальное время. Если Y — любая другая задача в NP, то $Y \leq_P X$, а следовательно, из (27.1) задача Y может быть решена за полиномиальное время. Отсюда $NP \subseteq P$; в сочетании с (27.11) приходим к искомому заключению. ■

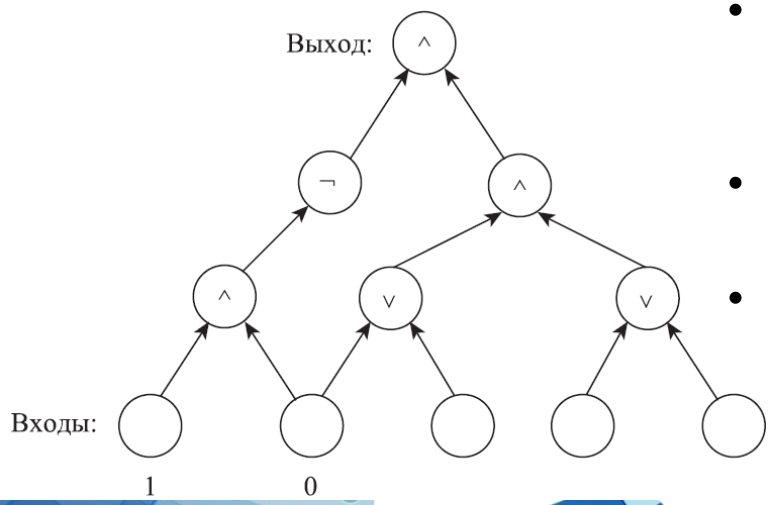
# Выполнимость булевой схемы: первая NP-полная задача

- Почему ∄ две несовместимые задачи X' и X'', для которых ∄ X ∈ NP с тем свойством, что X' $\leq_P$ X и X'' $\leq_P$ X?
- Почему в NP ∄ бесконечная последовательность задач $X_1, X_2, \ldots$, ∀ из которых строго сложнее предыдущей?

NP-полнота показывается не просто.ит  В 1971 году Кук и Левин независимо показали, как доказать NP-полноту задачи выполнимости булевой схемы (circuit satisfiability):

Рассмотрим стандартные булевы операторы: ∧ (И), ∨ (ИЛИ) и ¬ (НЕ).

Наше определение строится по образцу физической схемы, собранной из логических элементов, реализующих эти операторы. Затем булева схема K определяется как помеченный, направленный ациклический граф




- Источники в K (узлы, не имеющие входящих ребер) помечаются одной из констант — 0 или 1 либо именем конкретной переменной. Узлы последнего типа в дальнейшем называются входами схемы.
- Все остальные узлы помечаются одним из булевых операторов ∧, ∨ или ¬; узлы с меткой ∧ или ∨ имеют два входящих ребра, а узлы с меткой ¬ — одно.
- Также существует один узел, который не имеет выходящих ребер и представляет выход — результат, вычисляемый схемой.

**Пример.** Двум левым источникам присвоены значения 1 и 0, а следующие три источника соответствуют входам. Если подать на входы значения 1, 0, 1 (слева направо), то в логических элементах второй строки будут получены значения 0, 1, 1, в логических элементах третьей строки — значения 1, 1, и на выходе — значение 1.

# Выполнимость булевой схемы: первая NP-полная задача

**Задача выполнимости булевой схемы.** Для заданной схемы нужно решить, существует ли распределение значений по входам, для которого на выходе будет получено значение 1.

***Теорема 27.1(Кука и Левина).*** Задача выполнимости булевой схемы является NP-полной.

Как упоминалось выше, чтобы доказать (т27.1), следует рассмотреть произвольную задачу X в NP и показать, что X $\leq_P$ ***Выполнимость булевой схемы***.

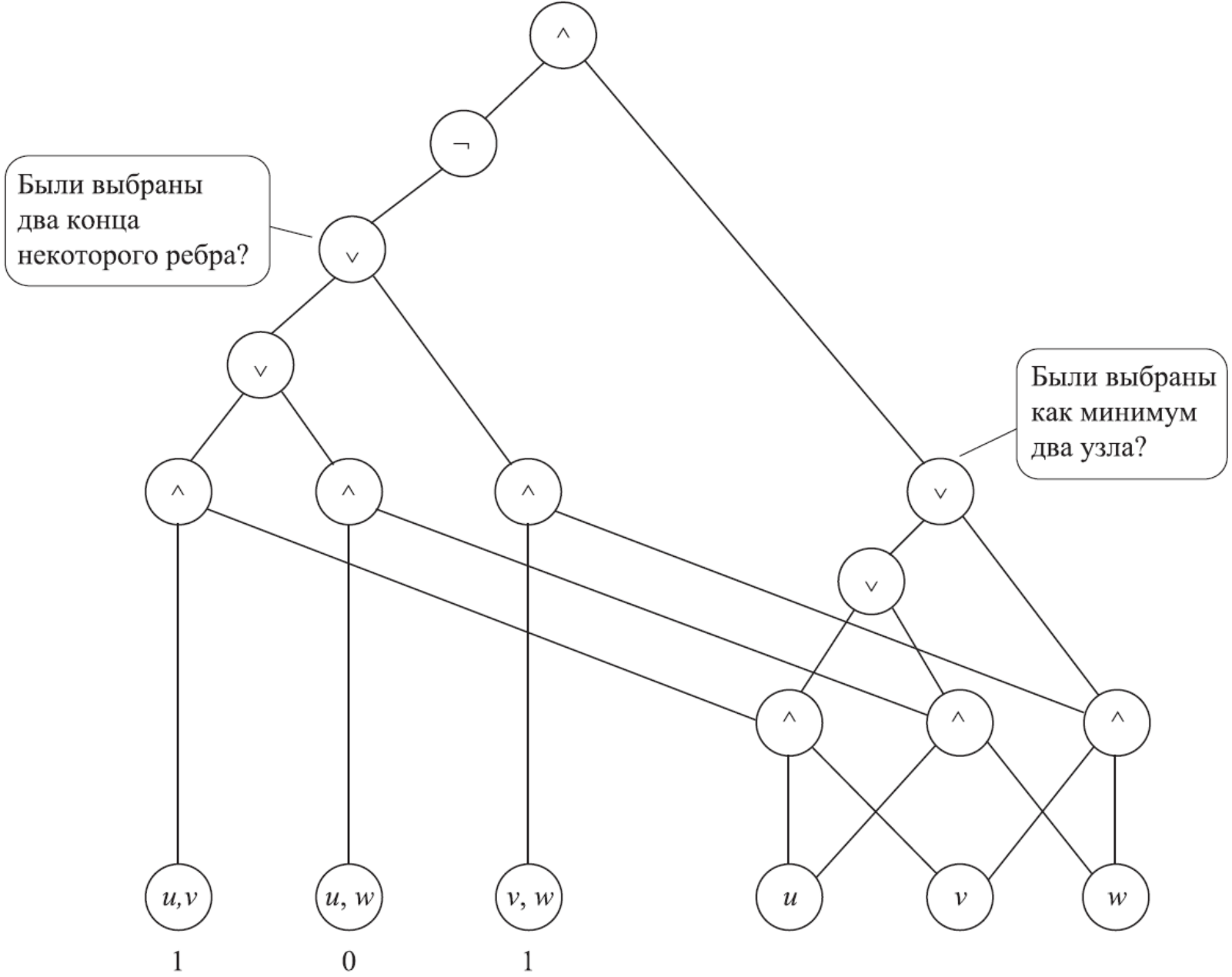
Основная идея доказательства. Мы используем факт, что любой алгоритм, который получает фиксированно n битов на входе и выдает ответ «да/нет», может быть представлен схемой только что определенного типа: такая схема эквивалентна алгоритму, то есть ее результат равен 1 для значений, для которых алгоритм выдает ответ «да». Более того, если алгоритм выполняется за серию шагов, полиномиальных по n, то схема имеет полиномиальный размер.

***Пример. Содержит ли заданный граф G независимое множество из двух узлов?***

Входные данные s представляют граф из n узлов, который будет задаваться $\binom{n}{2}$ битами: для каждой пары узлов будет присутствовать бит, указывающий, соединены ли эти два узла ребром. Сертификат t может задаваться n битами: для каждого узла включается бит, указывающий, принадлежит ли этот узел предлагаемому независимому множеству. Эффективный сертифицирующий алгоритм должен проверить два факта: что по крайней мере два бита в t равны 1 и что никакие два бита в t не равны 1, если они представляют два конца ребра (что определяется соответствующим битом в s). Теперь для конкретной длины n, соответствующей интересующим нас входным данным s, строится эквивалентная схема K.

# Пример

Предположим, к примеру, что мы хотим получить ответ на задачу для графа G с тремя узлами u, v, w, в котором узел v соединен с u и w. n = 3. (Фактически правая часть схемы проверяет, что были выбраны как минимум два узла, а левая — что не были выбраны оба конца любого ребра). Ребра G кодируются константами в первых трех источниках, а остальные три источника (представляющие узлы, включаемые в независимое множество) остаются переменными. Заметим, что выполнимость этого экземпляра задачи выполнимости схемы проверяется передачей 1, 0, 1 на вход. Это соответствует выбору узлов u и w, которые действительно образуют независимое множество из двух узлов в трехузловом графе G.




Схема, эквивалентная эффективному сертифицирующему алгоритму для нашей задачи с произвольным графом из трех узлов.

# Доказательство NP-полноты других задач

***Утверждение 27.13.*** Если Y — NP-полная задача, а X — задача в NP, обладающая свойством $Y \leq_P X$, то задача X является NP-полной.

Доказательство. Так как $X \in NP$, необходимо проверить только свойство (ii) определения. Пусть Z — любая задача в NP. Имеем $Z \leq_P Y$ вследствие NP-полноты Y и $Y \leq_P X$ по определению. Из (27.10) следует, что $Z \leq_P X$. ■

***Утверждение 27.14.*** Задача 3-SAT является NP-полной.

Доказательство. д./з.

***Утверждение 27.15.*** Все следующие задачи являются NP-полными: задача о независимом множестве, задача упаковки множества, задача о вершинном покрытии и задача покрытия множества.

Доказательство. Каждая из этих задач обладает тем свойством, что она принадлежит NP, и задача 3-SAT (а следовательно, и задача SAT) может быть сведена к ней. ■

***Основная стратегия доказательства NP-полноты новой задачи X выглядит примерно так:***

1. Доказать, что $X \in NP$.
2. Выбрать задачу Y, которая заведомо является NP-полной.
3. ***Доказать, что $Y \leq_P X$.*** Рассмотреть произвольный экземпляр $s_Y$ задачи Y и показать, как построить за полиномиальное время экземпляр $s_X$ задачи X, обладающий следующими свойствами.
   a. Если $s_Y$ является экземпляром Y, на который дается ответ «да», то и $s_X$ является экземпляром X, на который дается ответ «да».
   b. Если $s_X$ является экземпляром X, на который дается ответ «да», то и $s_Y$ является экземпляром Y, на который дается ответ «да».

Другими словами, тем самым устанавливается, что $s_Y$ и $s_X$ имеют одинаковый ответ.

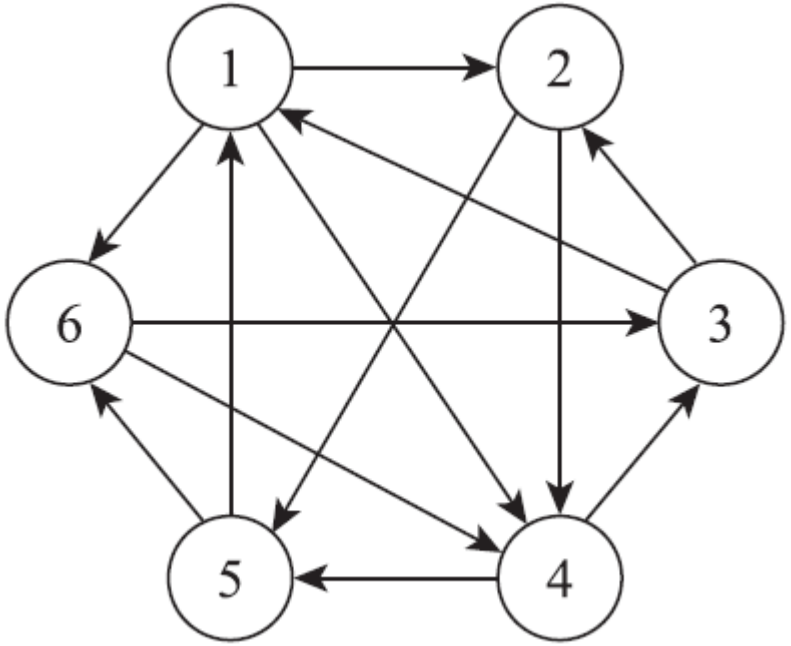
# Задачи упорядочения

Сейчас рассматривались задачи, которые (как, например, задачи о независимом множестве и вершинном покрытии) подразумевали перебор подмножеств набора объектов; также были представлены задачи (как, например, 3-SAT), основанные на поиске комбинаций 0/1 в наборе переменных. Еще один тип вычислительно сложных задач связан с поиском по множеству всех перестановок коллекции объектов.

# Задача о гамильтоновом цикле

У задачи коммивояжера имеется естественная аналогия, которая образует одну из фундаментальных задач в теории графов. Для заданного направленного графа G = (V, E) цикл C в графе G называется гамильтоновым циклом, если он посещает каждую вершину ровно один раз, — иначе говоря, он «обходит» все вершины без повторений. Например, направленный граф содержит несколько гамильтоновых циклов; в одном из них узлы посещаются в порядке 1, 6, 4, 3, 2, 5, 1, а в другом — в порядке 1, 2, 4, 5, 6, 3, 1.

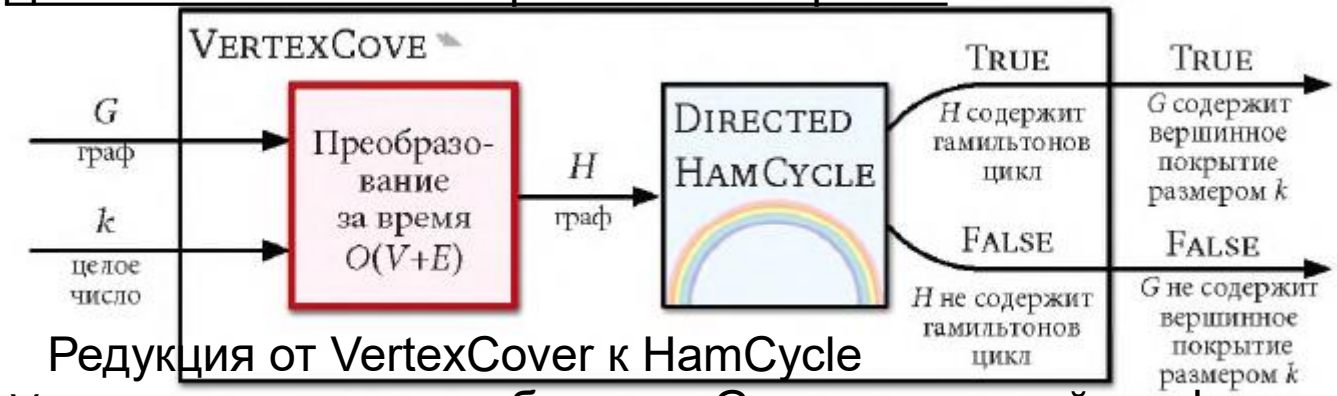
Задача о гамильтоновом цикле формулируется просто: Содержит ли заданный направленный граф G гамильтонов цикл?



# Доказательство NP-полноты задачи о гамильтоновом цикле

***Утверждение 27.16.*** Задача о гамильтоновом цикле является NP-полной.

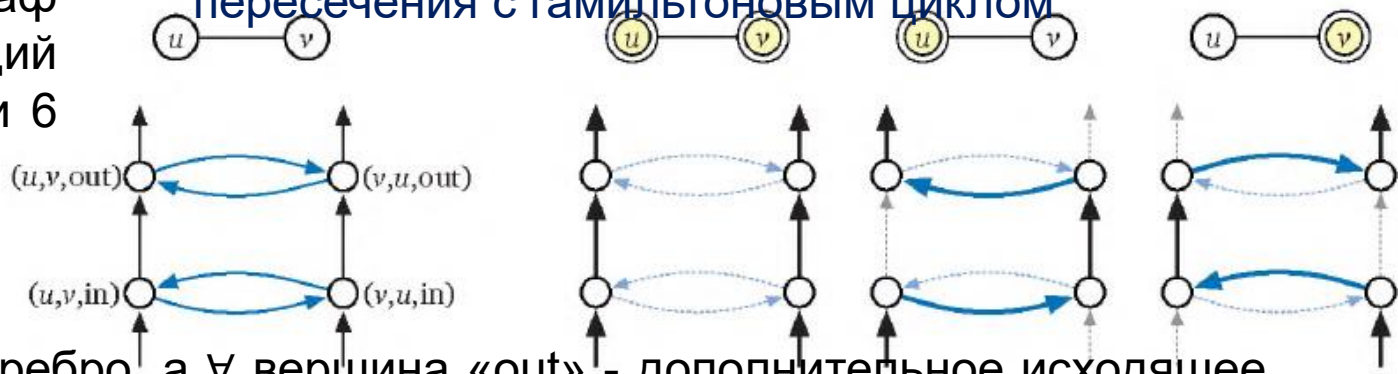
Доказательство. От вершинного покрытия




Если задан ненаправленный граф G и целое число k, то мы формируем направленный граф H такой, что H содержит гамильтонов цикл, если и только если G содержит вершинное покрытие размером k.

Редукция от VertexCover к HamCycle

Гаджет ребра и только возможные его пересечения с гамильтоновым циклом




∀ ненаправленного ребра *uv* в G направленный граф H содержит гаджет ребра (edge gadget), состоящий из 4 вершин $(u, v, in), (u, v, out), (v, u, in), (v, u, out)$ и 6 направленных ребер: $(u, v, in) \to (u, v, out)$, $(u, v, in) \to (v, u, in), (v, u, in) \to (u, v, in), (v, u, in) \to (v, u, out), (u, v, out) \to (v, u, out), (v, u, out) \to (u, v, out)$,

∀ вершина «in» имеет дополнительное входящее ребро, а ∀ вершина «out» - дополнительное исходящее ребро. ∀ гамильтонов цикл в H обязательно должен проходить через гаджет ребра по одному из трех путей - либо прямо, либо в обход от одной стороны к другой и обратно. Эти варианты будут соответствовать обеим вершинам u и v, только u или только v, принадлежащим к одному вершинному покрытию.

# Доказательство NP-полноты задачи о гамильтоновом цикле

Для каждой вершины и в G все гаджеты ребер для инцидентных ребер *uv* соединены в H в единый направленный путь, который мы называем цепь, вершин *(vertex chain)*.

Предположим, что вершина и имеет d соседей $v_1, v_2, \dots, v_d$. Тогда H содержит d - 1 дополнительных ребер (u, v, out) -> (u, v, in) ∀ i от 1 до d - 1

Наконец, H также содержит k вершинных покрытий (cover vertices) $x_1, x_2, \dots, x_k$. Каждое вершинное покрытие содержит направленное ребро к первой вершине в каждой цепи вершин и направленное ребро от последней вершины в каждой цепи вершин.

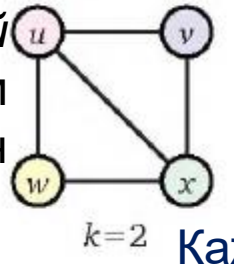
**Доказываем => .** Предположим, $C = \{u_1, u_2 \dots u_k\}$ - вершинное покрытие графа G размером k.

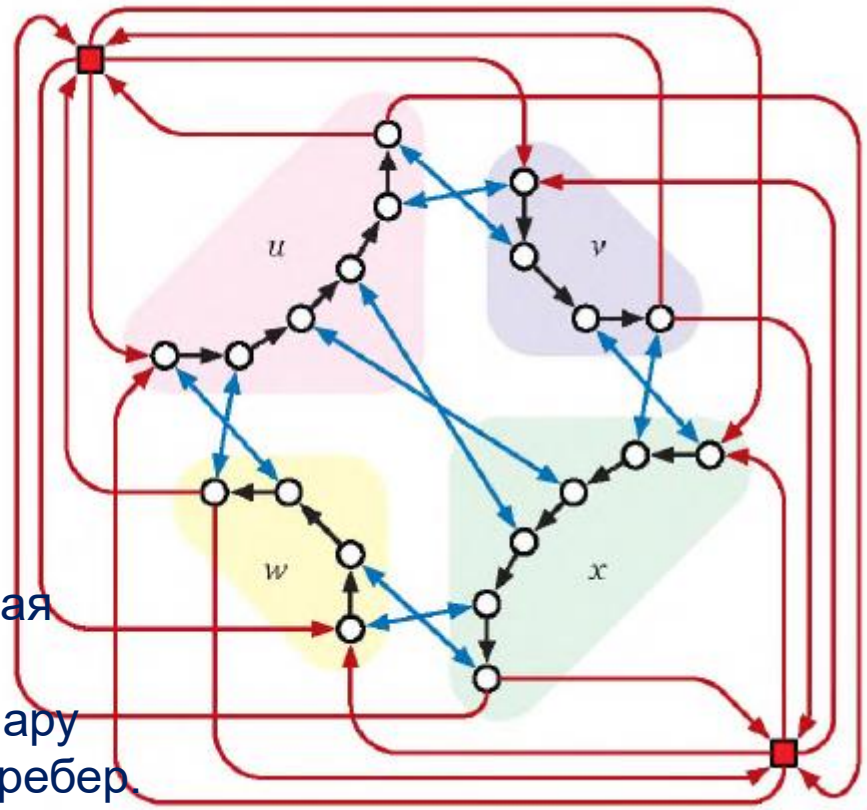
*Создадим гамильтонов цикл в H, который «кодирует» C.* ∀ индекса i от 1 до k мы проходим путь из вершинного покрытия x через цепь вершин для $u_i$ к вершинному покрытию $x_{i+1}$ ($x_1$ при i = k).

При проходе цепи ∀ вершины $u_i$ мы определяем, как продолжить движение из каждого узла (u, v, in):

- если v ∈ C, то следовать по ребру ($u_i$, v, in) → ($u_i$, v, out);
- если v ∉ C, то выполнить обход через ($u_i$, v, in) → (v, $u_i$, in) → (v, $u_i$, out) → ($u_i$, v, out).

Пример редукции от VertexCover к DirectedHamCycle.

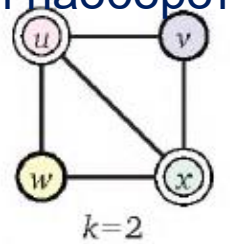


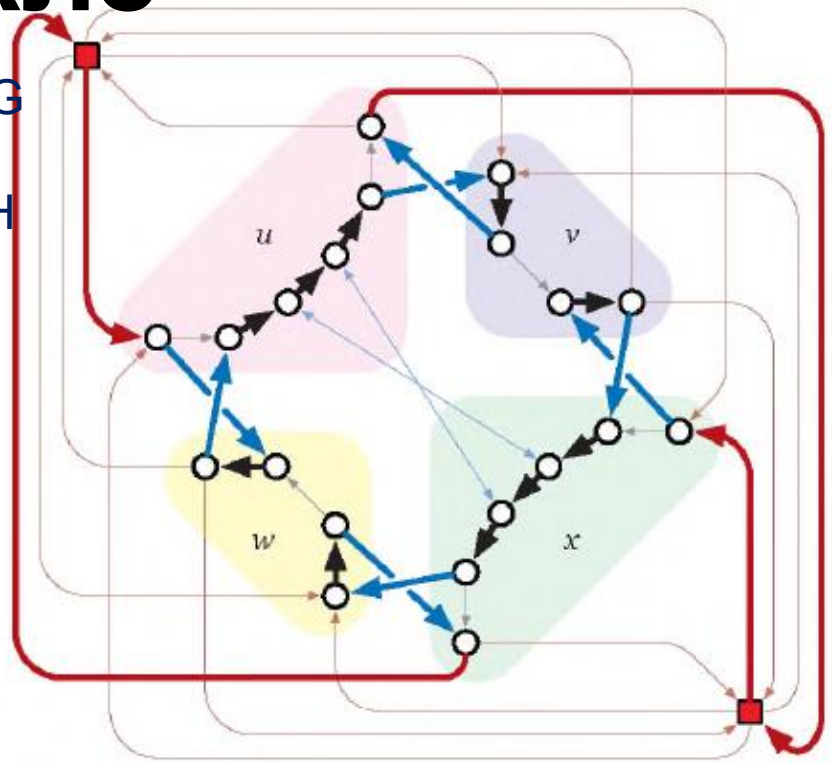




Каждая двунаправленная синяя стрелка представляет пару направленных ребер.

# Доказательство NP-полноты задачи о гамильтоновом цикле

Каждое вершинное покрытие размера k в G соответствует гамильтонову циклу в H и наоборот




Таким образом, ∀ ребра *uv* в G гамильтонов цикл посещает (u, v, in) и (и, v, out) как часть цепи вершин u, если u ∈ C, иначе – как часть цепи вершин v.

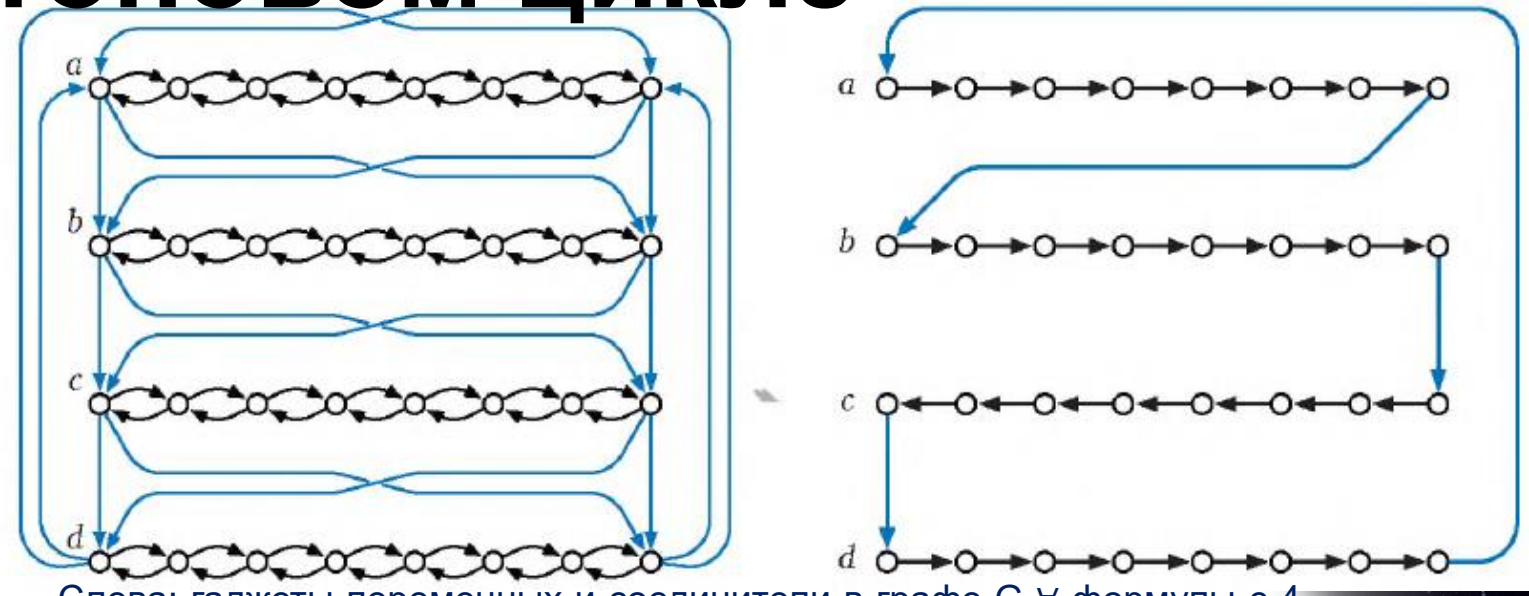
**Доказываем <=** С другой стороны, предположим, что H содержит гамильтонов цикл C. Этот цикл обязательно должен содержать ребро от ∀ вершинного покрытия к началу некоторой цепи вершин. В нашем случае анализ гаджетов ребер индуктивно предполагает, что после того, как C входит в цепь вершин для некоторой вершины и, он обязательно должен пройти всю цепь вершин. В частности, в каждой вершине (u, v, in) цикл обязательно должен содержать либо одно ребро (u, v, in) → (u, v, out), либо путь обхода (u, v, in) → (v, u, in) → (v, u, out) → (u, v, out), за которым следует ребро к следующему гаджету ребра в цепи вершин и или к вершинному покрытию, если это последний гаджет ребра в цепи вершин и. В частности, если C содержит ребро обхода (u, v, in) -> (v, u, in), то оно не может содержать ребра между любым вершинным покрытием и цепью вершин v. Из этого следует, что C проходит ровно к цепей вершин. Кроме того, эти цепи вершин описывают вершинное покрытие исходного графа G, так как C посещает вершину (u, v, in) для каждого ребра uv в графе G.

Преобразование из G в H выполняется за время $O(V^2)$.

# Доказательство NP-полноты задачи о гамильтоновом цикле

Доказательство. От 3SAT

Если задана произвольная формула 3CNF Ф с n переменными $x_1, x_2, \dots, x_n$ и k компонентами $c_1, c_2, \dots, c_k$, то мы создаем направленный граф H, который содержит гамильтонов цикл, если и только если формула Ф выполнима, как показано ниже. ∀ x мы формируем гаджет переменной, состоящий из двусвязного списка 2k вершин (i, 0), (i, 1), …, (i, 2k), соединенных ребрами $(i, j-1) \to (i, j)$ и $(i, j) \to (i, j-1)$ для каждого индекса j.




Слева: гаджеты переменных и соединители в графе G ∀ формулы с 4 переменными и 4 компонентами. Справа: гамильтонов цикл в G, соответствующий присваиванию a = b = d = True и c = False

Соединим первый и последний узел в ∀ смежной паре гаджетов переменных дополнительными ребрами:

$$(i, 0) \to (i+1, 0), (i, 2k) \to (i+1, 0), (i, 0) \to (i+1, 2k)\ (i, 2k) \to (i+1, 2k)\ \forall i;$$

мы также соединяем конечные точки первого и последнего гаджета переменной ребрами:

$$(n, 0) \to (1, 0), (n, 2k) \to (1, 0), (n, 0) \to (1, 2k), (n, 2k) \to (1, 21).$$

Полученный в итоге граф G содержит ровно $2^n$ гамильтоновых циклов, по одному для каждого присваивания логических значений n переменных в формуле Ф. В частности, ∀ i мы проходим i-й гаджет переменной слева направо, если x = True, и справа налево, если x, = False.

# Доказательство NP-полноты задачи о гамильтоновом цикле

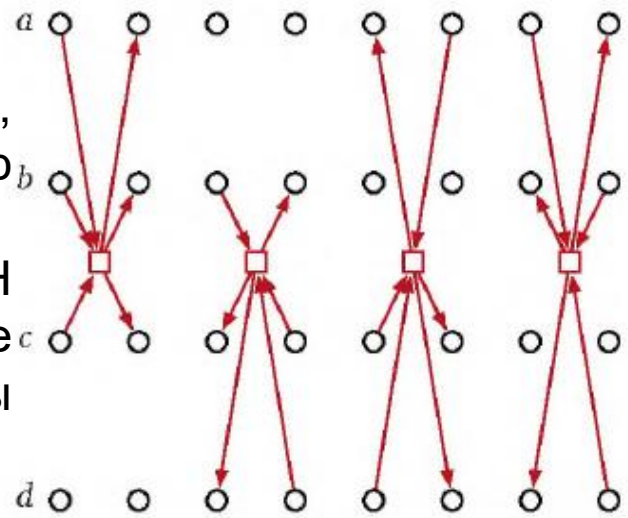
Теперь мы расширяем G до более крупного графа H, добавляя вершину компоненты (clause vertex) [j] для каждой компоненты $c_j$, соединенную с гаджетами переменных шестью ребрами, как показано на рис. ∀ положительного литерала $x_i$ в $c_j$ добавим ребра $(i, 2j-1) \to [i] \to (i, 2j)$, а для каждого отрицательного литерала $x_i$ в $c_j$ - ребра $(i, 2j) \to [j] \to (i, 2j-1)$. Соединения с вершинами компонент гарантируют, что гамильтонов цикл в G может быть расширен до гамильтонова цикла в H, если и только если соответствующее присваивание переменным делает выполнимой формулу Φ. Теперь анализ исчерпывающего случая предполагает, что H содержит гамильтонов цикл, если и только если формула Φ выполнима.

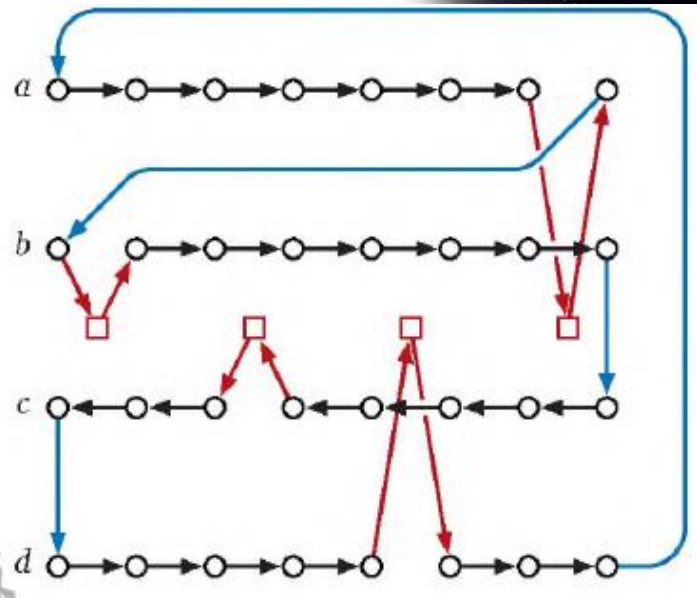
Слева: гаджеты компонент для формулы

$$(a \vee b \vee c)\,(b \vee \bar{c} \vee \bar{d})\,(a \vee c \vee d)\,(a \vee \bar{b} \vee \bar{d}).$$

Справа: гамильтонов цикл в графе H, соответствующий выполнимому присваиванию a = b = d = True и c = False

Преобразование формулы Φ в граф H выполняется за время O(kn), которое не более чем квадратично относительно общей длины формулы



# Расширения: задача о гамильтоновом пути

Для заданного направленного графа G = (V, E) путь P в G называется гамильтоновым путем, если каждая вершине входит в него ровно один раз. Такой путь состоит из разных узлов $v_{i_1}, v_{i_2}, \dots, v_{i_n}$ в определенном порядке, образующих все множество вершин V; в отличие от гамильтонова цикла, наличие ребра из $v_{i_n}$ обратно в $v_{i_1}$ не обязательно. Задача о гамильтоновом пути формулируется так:

Содержит ли заданный направленный граф G гамильтонов путь?

***Утверждение 27.17.*** Задача о гамильтоновом пути является NP-полной.

# Расширения:

Кратчйший гамильтонов цикл со взвешенными ребрами

*Обобщенная концепция расстояния:* ∀ упорядоченной пары городов $(v_i,v_j)$ задается $d(v_i,v_j) \geq 0$, которое считается расстоянием от $v_i$ до $v_j$. От расстояний не требуется ни симметричность (может быть $d(v_i,v_j) \neq d(v_j,v_i)$), ни выполнение неравенства треугольника (сумма $d(v_i,v_j)$ и $d(v_j,v_k)$ может быть меньше «прямого» расстояния $d(v_i,v_k)$).

***Формулировка задачи коммивояжера применяется:***

- планирование оптимальных перемещений манипулятора;
- для обслуживания запросов ввода/вывода к диску;
- для упорядочения выполнения n программных модулей для минимизации времени переключения контекста.

**Формулировка:** Для заданного множества расстояний требуется упорядочить города в маршрут $v_{i_1}, v_{i_2}, \dots, v_{i_n}$, с $i_1 = 1$, чтобы свести к минимуму общее расстояние $\sum_j d(v_{i_j}, v_{i_{j+1}}) + d(v_{i_n}, v_{i_1})$. Требование $i_1 = 1$ просто «ориентирует» маршрут так, чтобы он начинался в исходном городе, а слагаемые в сумме просто задают расстояние от текущего города в маршруте до следующего.

Версия с принятием решения:

*Для заданного набора расстояний между n городами и границы D существует ли маршрут, длина которого не превышает D?*

***Утверждение 27.18.*** Задача коммивояжера является NP-полной.

# Задача о трехмерном сочетании

**Фундаментальные задачи о разбиении** - в которых ищутся способы разбиения коллекции объектов на подмножества.

Одна из - задача о трехмерном сочетании, которая может считаться усложненной версией задачи о двудольном паросочетании. Задача о двудольном паросочетании может рассматриваться следующим образом: даны два множества X и Y, каждое из которых имеет размер n, и множество P пар из X × Y. Вопрос: существует ли в P такое множество из n пар, в котором каждый элемент в X ∪ Y содержится ровно в одной паре? Связь с задачей двудольного паросочетания очевидна: множество P таких пар просто соответствует ребрам двудольного графа.

***Рассмотрим следующую задачу о трехмерном сочетании:***

Для заданных непересекающихся множеств X, Y и Z, каждое из которых имеет размер n, и заданного множества T ⊆ X × Y × Z упорядоченных триплетов существует ли в T такое множество из n триплетов, что каждый элемент X ∪ Y ∪ Z содержится ровно в одном из этих триплетов?

Такой набор триплетов называется идеальным трехмерным сочетанием.

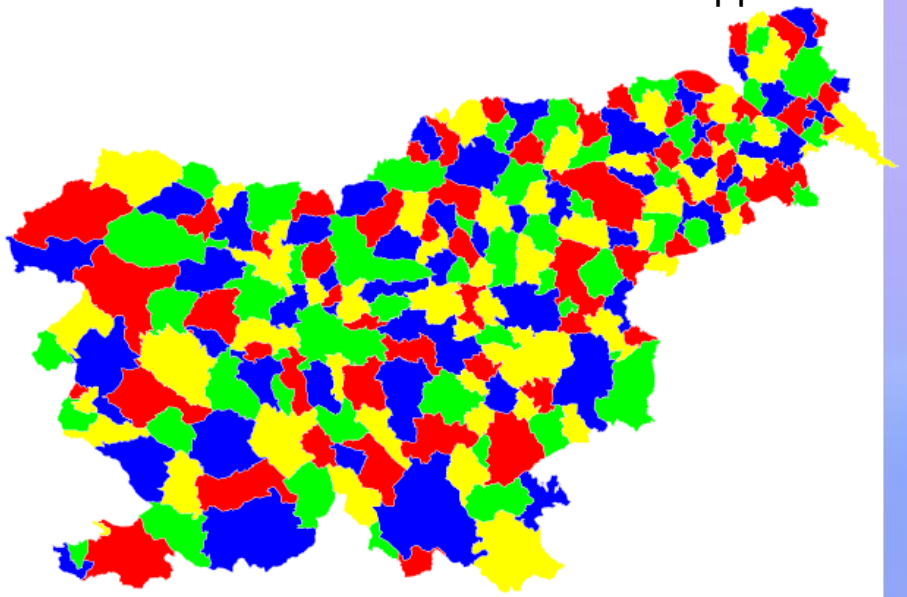
Конкретнее, задача о трехмерном сочетании является частным случаем задачи покрытия множества, поскольку мы ищем покрытие универсального множества U = X ∪ Y ∪ Z с использованием не более n множеств из заданного набора (триплетов). Приведенные выше рассуждения легко преобразуются в доказательства того, что ***Трехмерное сочетание*** $\leq_P$ ***Покрытие множества*** и **Трехмерное сочетание** $\leq_P$ **Упаковка множества**.

***Утверждение 27.19.*** Задача о трехмерном сочетании является NP-полной. Доказательство. дз

# Задачи о разбиении

При раскрашивании карты (например, стран на карте мира) желательно раскрасить соседние области в разные цвета, чтобы их границы были четко видны, — но при этом, чтобы избежать лишней пестроты, использовать минимальное количество цветов.

В середине XIX века ***Фрэнсис Гатри*** заметил, что карту графств Англии можно раскрасить таким образом всего четырьмя цветами, и его заинтересовало, обладает ли этим свойством любая карта. Он спросил своего брата, который передал вопрос одному из своих профессоров; так родилась знаменитая математическая задача: гипотеза четырех цветов.



# Задача о раскраске графа

Под раскраской графа понимается аналогичный процесс с ненаправленным графом G, в котором узлы играют роль раскрашиваемых областей, а ребра представляют соседние пары. Требуется назначить цвет каждому узлу G так, чтобы при наличии ребра (u, v) узлам u и v были назначены разные цвета; целью является выполнение этих условий с минимальным набором цветов. В более формальном варианте k-раскраской G называется такая функция $f:V \rightarrow \{1, 2, ..., k\}$, что для каждого ребра (u, v) выполняется f (u) = f (v). (Таким образом, цвета обозначаются 1, 2, ..., k, а функция f представляет выбор цвета для каждого узла.)

Если для G существует k-раскраска, он называется ***k-раскрашиваемым***.

В отличие от карт на плоскости, вполне очевидно, что не существует фиксированной константы k, с которой любой граф имеет k-раскраску: например, если взять множество из n узлов и соединить каждую пару ребром, то для раскраски полученного графа потребуется n цветов. Тем не менее алгоритмическая версия задачи очень интересна:

Для заданного графа G и границы k имеет ли граф G k-раскраску?

Будем называть эту задачу задачей раскраски графа — или k-раскраски, когда мы хотим подчеркнуть конкретный выбор k.

# Применения задачи о раскраске графа

Задача раскраски графа находит очень широкий диапазон применения. Хотя реальная потребность в ней в картографии пока неочевидна, эта задача естественным образом возникает в ситуациях с распределением ресурсов при наличии конфликтов.

- ***При распределении ресурсов ОС.*** Допустим, имеется набор из n процессов в системе, которая позволяет выполнять несколько заданий в параллельном режиме. При этом некоторые пары заданий не могут выполняться одновременно, потому что им нужен доступ к некоторому ресурсу. Для следующих k временных квантов требуется спланировать выполнение процессов, чтобы каждый процесс выполнялся минимум в одном из них. Возможно ли это? Если построить граф G на базе множества процессов, соединяя два процесса ребром при наличии конфликта, то k-раскраска G будет представлять план выполнения, свободный от конфликтов: все узлы, окрашенные в цвет j, будут выполняться на шаге j, а вся конкуренция за ресурсы будет исключена.
- ***При разработке компиляторов.*** Предположим, при компиляции программы мы пытаемся связать каждую переменную с одним из k регистров. Если две переменные используются одновременно, они не могут быть связаны с одним регистром (в противном случае присваивание одной переменной приведет к потере значения другой). Для множества переменных строится граф G; две переменные соединяются ребром в том случае, если они используются одновременно. K-раскраска G соответствует безопасному способу распределения переменных между регистрами: все узлы с раскраской j могут быть связаны с регистром j, так как никакие два из них не используются одновременно.
- ***При распределении частот для беспроводной связи.*** Требуется назначить одну из k частот каждому из n устройств; если два устройства находятся достаточно близко друг к другу, им должны быть присвоены разные длины волн для предотвращения помех.

# Вычислительная сложность задачи о раскраске графа

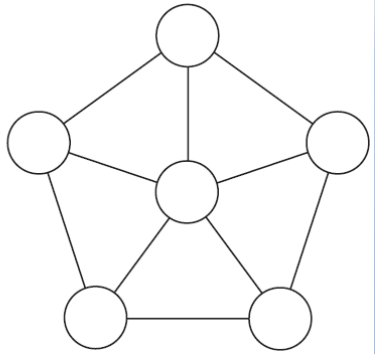
Какую сложность имеет задача k-раскраски?

***Утверждение 27.20.*** Граф G является 2-раскрашиваемым в том, и только в том случае, если он является двудольным.

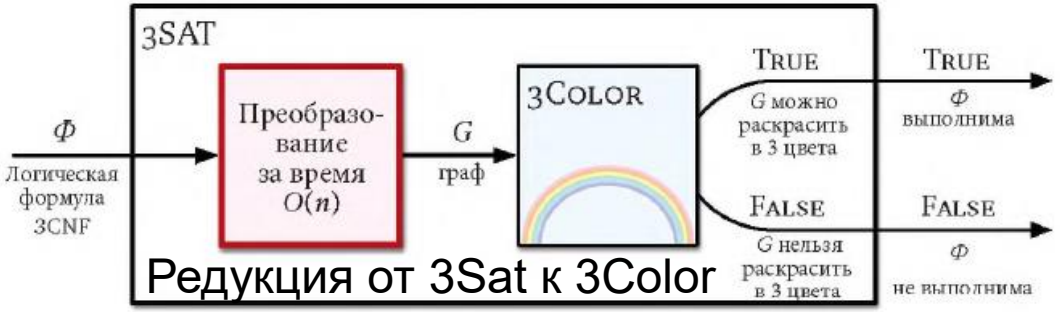
Это означает, что алгоритм может использоваться для принятия решения о том, является ли входной граф 2-раскрашиваемым, за время O(m + n), где n — количество узлов G, а m — количество ребер.

Стоит перейти к k = 3 цветам, как ситуация значительно усложняется. Никакого простого эффективного алгоритма для задачи 3-раскраски не видно, и рассуждать об этой задаче вообще сложно. Например, изначально может возникнуть впечатление, что в любом графе, не являющиеся 3-раскрашиваемым, присутствует «доказательство» в форме четырех узлов, являющихся взаимно смежными (для которых потребуются четыре разных цвета) — но это не так.

Корректная k-раскраска (proper k-coloring) графа G = (V, E) - это функция C: V → {1, 2, …, k], которая присваивает один из k «цветов» каждой вершине так, что каждое ребро имеет два различных цвета в своих конечных точках. (В действительности «цветами» могут быть произвольные метки, которые для упрощения мы представляем небольшими целыми числами, а неэлектромагнитными частотами, векторами CMYK или числовыми значениями «Пантон», например.)



# Доказательство NP-полноты задачи о 3-раскраске



Редукция от 3Sat к 3Color

***Утверждение 27.21.*** Задача о 3-раскраске является NP-полной.

Доказательство. Используем редукцию от 3Sat. (Почему 3 Sat? Потому что в названии присутствует 3. И это не шутка.)

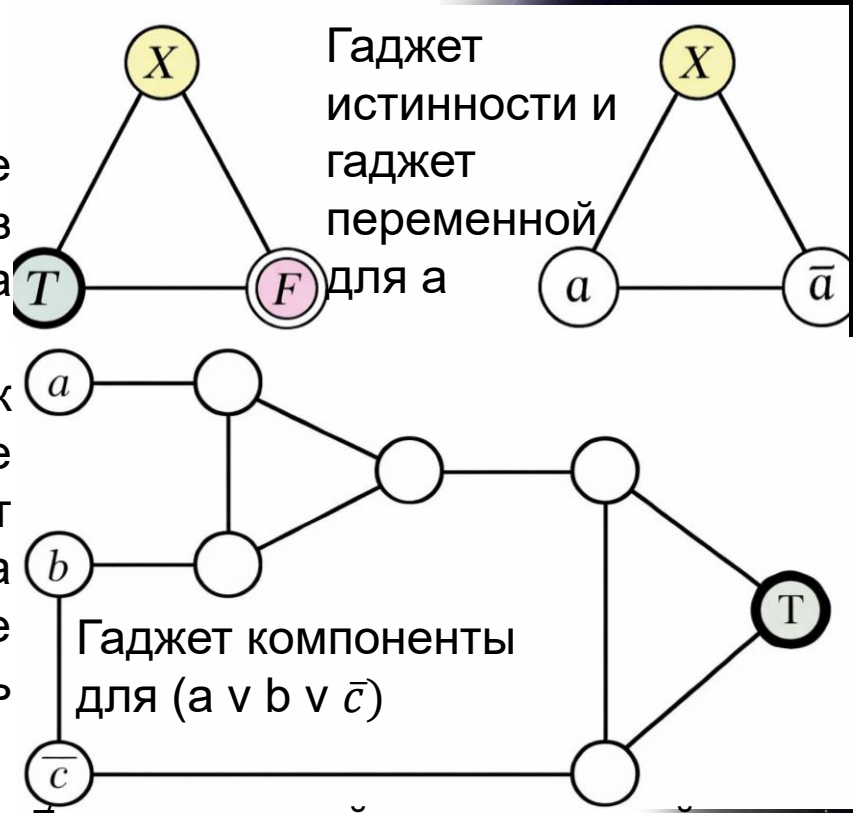
Опишем редукцию, используя стандартную стратегию разбиения выходного графа G на гаджеты (gadgets), подграфы, которые поддерживают различные семантики (смысловые значения) исходной формулы Φ в языке раскраски графов. Разбиение редукции на отдельные гаджеты удобно как для понимания существующих редукций и доказательства их корректности, так и для проектирования новых NP-трудных редукций. Наша редукция формула Φ граф использует три типа гаджетов:

- ∃! гаджет истинности (truth gadget): треугольник стремя вершинами T, F и X, с понятными обозначениями для True, False и Other соответственно. Поскольку все эти вершины соединены, они непременно должны иметь различные цвета в любой трехцветной раскраске. Для удобства будем называть эти цвета True, False и Other. Таким образом, когда мы говорим, что узел окрашен в цвет True, то подразумеваем, что он имеет тот же цвет, что и вершина T;
- ∀ переменной a граф содержит гаджет переменной (variable gadget), представленный треугольником, соединяющим два новые узла, помеченные a и $\bar{a}$, с узлом X в гаджете истинности. Узел a обязательно должен быть окрашен в цвет True или False, следовательно, узел $\bar{a}$ обязательно должен быть окрашен в цвет False или True соответственно;

# Доказательство NP-полноты задачи о 3-раскраске

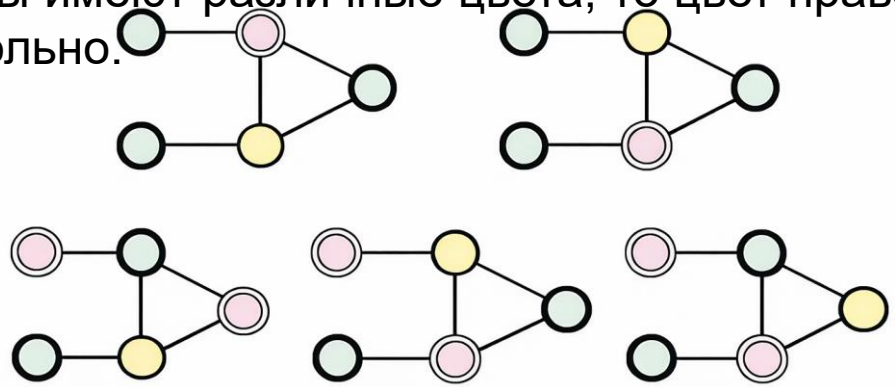
- ∀ компоненты в формуле Ф граф содержит гаджет компоненты (clause gadget). Каждый гаджет компоненты соединяет три узла литералов (из соответствующих гаджетов переменных) с узлом T (из гаджета истинности), используя пять новых непомеченных узлов и десять ребер.

По существу, каждый треугольник в гаджете компоненты ведет себя как «мажоритарный логический элемент». В любой допустимой схеме трехцветной раскраски если две вершины слева от треугольника имеют одинаковый цвет, то правая крайняя вершина этого треугольника обязательно должна иметь тот же цвет. С другой стороны, если две левые вершины имеют различные цвета, то цвет правой вершины можно выбрать произвольно.




Гаджет истинности и гаджет переменной для a

Гаджет компоненты для (a v b v $\bar{c}$)



Все допустимые схемы трехцветной раскраски «полугаджета», вплоть до перестановок цветов

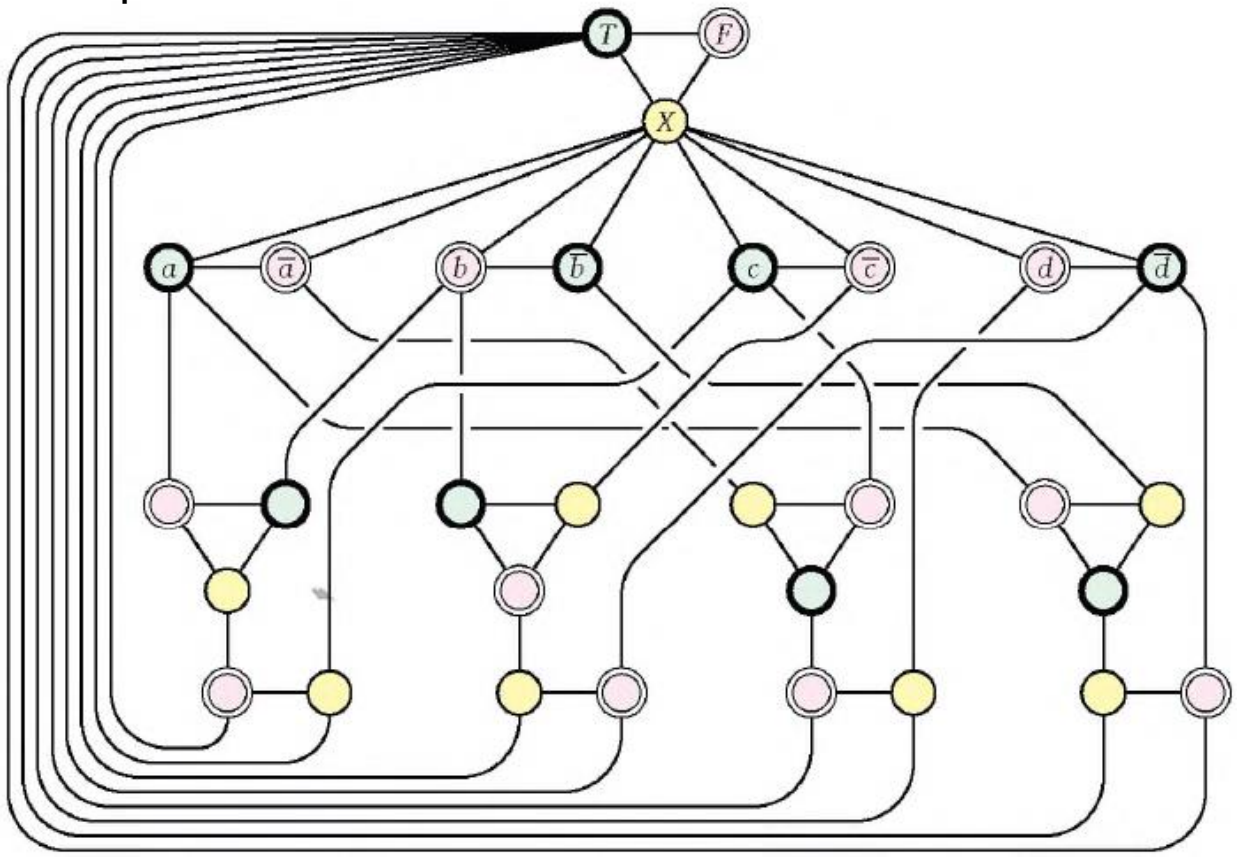
Из этого следует, что ∄ допустимой трехцветной раскраски гаджета компоненты, в которой все три узла литералов окрашены в цвет False. С другой стороны, любая раскраска узлов литералов более чем одним цветом может быть расширена до допустимой трехцветной раскраски гаджета компоненты.

# Доказательство NP-полноты задачи о 3-раскраске

Гаджеты переменных принуждают каждый узел литерала иметь цвет True или False, следовательно, в любой допустимой трехцветной раскраске гаджета компоненты как минимум один узел литерала окрашен в цвет True. Итоговый граф G содержит ровно один узел T, ровно один узел F и ров-но два узла a и a для каждой переменной.

Например, на рис. показан граф, полученный из той же формулы 3CNF $(a \vee b \vee c) \wedge (b \vee c \vee \bar{d}) \wedge (\bar{a} \vee c \vee d) \quad \wedge (a \vee \bar{b} \vee \bar{d})$ . Эта трехцветная раскраска является одним из нескольких вариантов, соответствующих выполнимому присваиванию a = c = True, b = d = False.

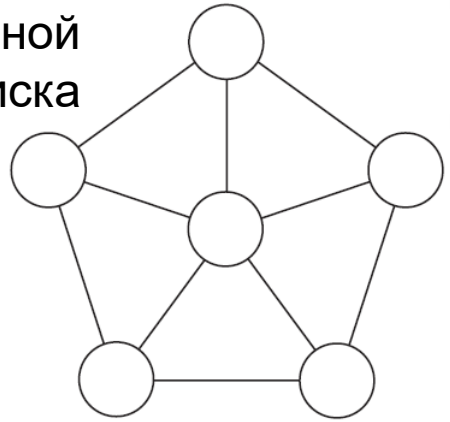
Если формула выполнима, то мы можем раскрасить узлы литералов в соответствии с любым выполнимым присваиванием, а затем (поскольку каждая компонента выполнима) распространить раскраску на каждый гаджет компоненты. С другой стороны, если граф можно раскрасить в три цвета, то мы можем извлечь выполнимое присваивание из любой схемы трехцветной раскраски - как минимум один из трех узлов литералов в каждом гаджете компоненты окрашен в цвет True.



# Заключение: о проверке гипотезы четырех цветов

Поскольку 3Color является особым случаем более общей задачи раскраски графа, - каково минимальное число цветов? - эта более общая задача оптимизации также является NP.

В завершение этого раздела стоит рассказать и о том, как закончилась история гипотезы четырех цветов для карт на плоскости. Более чем через 100 лет гипотеза была доказана Аппелем и Хакеном в 1976 году. Структура доказательства была простой индукцией по количеству областей, но шаг индукции включал около двух тысяч относительно сложных случаев, проверку которых пришлось поручить компьютеру. Многие математики были недовольны таким результатом: они надеялись получить доказательство, которое бы давало представление о том, почему гипотеза была истинной, — а вместо этого получили перебор запредельной сложности, проверку которого нельзя было осуществить вручную. Задача поиска относительно короткого, доступного доказательства все еще остается открытой.



# Численные задачи

А теперь рассмотрим некоторые вычислительно сложные задачи, основанные на арифметических операциях с числами. Как вы увидите, вычислительная неразрешимость в данном случае обусловлена способом кодирования некоторых из представленных ранее задач в представлении очень больших целых чисел.

# Задача о суммировании подмножеств

Базовой задачей в этой категории будет задача о суммировании подмножеств — частный случай задачи о рюкзаке. Версия этой задачи с принятием решения может быть сформулирована следующим образом:

Задано множество натуральных чисел $w_1$, ..., $w_n$ и целевое число W. Существует ли подмножество $\{w_1, ..., w_n\}$, сумма которого равна ровно W?

В более общем виде эту проблему можно сформулировать, так как целые числа обычно будут задаваться в двоичном представлении, величина W в действительности экспоненциально зависит от размера входных данных; наш алгоритм не был алгоритмом с полиномиальным временем.

Эта проблема встречается во многих ситуациях:

- Алгоритмы сетевого потока, в которых пропускные способности целочисленные.
- Безопасность криптографической системы RSA обусловлена сложностью факторизации числа из 1000 битов. Но если бы время выполнения из $2^{1000}$ шагов считалось приемлемым, факторизация такого числа не создавала бы никаких трудностей.

Итак, основной вопрос: возможно ли решение задачи о суммировании подмножеств алгоритмом с (действительно) полиномиальным временем? Другими словами, существует ли алгоритм с временем выполнения, полиномиальным по n и log W? Или полиномиальным только по n?

# Доказательство NP-полноты задачи о суммировании подмножеств

***Утверждение 27.22.*** Задача о суммировании подмножеств является NP-полной.
Доказательство. Снова применим редукцию от задачи VertexCover.
Задан граф G и целое число k, нам необходимо вычислить множество чисел X ∈ ℕ и число T ∈ ℤ такое, что X содержит подмножество, сумма элементов которого равна T, если и только если G содержит вершинное покрытие размером k. Наше преобразование использует только два типа «гаджетов» - целые числа, представляющие вершины и ребра графа G.
Пронумеруем ребра в G произвольно от 0 до E - 1. Наше множество X содержит целое число b := $4^i$ для каждого ребра i и целое число

$$a_v := 4^E + \sum_{i \epsilon \Delta(v)} 4^i$$

∀ вершины v, где Δ(v) - множество ребер, для которых v является конечной точкой. С другой стороны, можем считать ∀ целое число в X (E + 1)-разрядным числом, записанным по основанию 4. Каждый E-й разряд равен 1, если целое число представляет вершину, и 0 в противном случае. ∀ i < E i-й разряд равен 1, если целое число представляет ребро i или одну из его конечных точек, и 0 в противном случае. После этого устанавливаем целевую сумму:

$$T := k4^E + \sum_{i=0}^{E-1} 2 \cdot 4^i$$

# Доказательство NP-полноты задачи о суммировании подмножеств

Теперь докажем, что эта редукция корректна.

**=>** Сначала предположим, что G содержит вершинное покрытие C размером k. Рассмотрим подмножество $X' := \{a_v | \, v \, C\} \, \epsilon \, \{b_i \mid \text{ребро i имеет ровно одну конечную точку в } C\}$.

Сумма элементов X', записанная по основанию 4, содержит самый старший разряд k, а все остальные разряды равны 2. Следовательно, сумма элементов X' в точности равна T.

**<=** С другой стороны, предположим, что существует подмножество X’ ⊆ X, сумма элементов которого равна T. В частности, мы обязательно имеем

$$\sum_{v \in V'} a_v + \sum_{i \epsilon E'} b_i = t$$

для некоторых подмножеств V' ⊆ V и E' ⊆ E. И снова если мы просуммируем эти числа по основанию 4, то не существует переносов в первых E разрядах, так как для каждого i существует только три числа в X, i-е разряды которых равны 1. Каждое число ребра b, вносит только одну 1 в i-й разряд суммы, но i-й разряд t равен 2. Следовательно, для каждого ребра в G как минимум одна из его конечных точек обязательно должна находиться в V'. Другими словами, V' является вершинным покрытием. С другой стороны, только числа вершин больше $4^E$ и |T/ $4^E$] = k, поэтому V' содержит не более к элементов. (В действительности легко видеть, что V содержит ровно k элементов.)

# Доказательство NP-полноты задачи о суммировании подмножеств

Например, если задан граф с четырьмя вершинами G = (V, E), где V = {u, v, w, x} и 1 = {uv, uw, vx, wx}, то наше множество X может содержать следующие числа с основанием 4:

$$a_u := 111000_4 = 1344$$
$$a_v := 110110_4 = 1300$$
$$a_w := 101101_4 = 1105$$
$$a_x := 100011_4 = 1029$$

$$b_{uv} := 010000_4 = 256$$
$$b_{uw} := 001000_4 = 64$$
$$b_{vw} := 000100_4 = 16$$
$$b_{vx} := 000010_4 = 4$$
$$b_{wx} := 000001_4 = 1$$

Если мы ищем вершинное покрытие размером k = 2, то наша целевая сумма должна быть равна T := 222222, = 2730. Вершинное покрытие {v, w} соответствует множеству $\{a_v, a_w, b_{uv}, b_{uw}, b_{vx}, b_{wx}\}$, сумма которого равна 1300+1105+256+64+4+1 = 2730. Эта редукция явно может быть выполнена за полиномиальное время.

***Да будет осмотрителен выполняющий редукцию!***

Здесь необходимо отметить одну тонкость. Алгоритм динамического программирования для решения задачи SubsetSum за время O(nT) когда-то был. Разве этот алгоритм не выполняется за полиномиальное время? Разве мы только что не доказали, что P = NP? Увы, жизнь не настолько проста. Действительно, время выполнения этого алгоритма является полиномиальной функцией от переменных n и T, но для точного определения как истинного алгоритма с полиномиальным временем выполнения это время обязательно должно быть полиномиальной функцией от размера входных данных (input size), т. е. от числа битов, требуемых для представления входных данных.

# Расширения: сложность некоторых задач планирования

Сложность задачи о суммировании подмножеств может использоваться для установления сложности целого класса задач планирования, включая некоторые задачи, не связанные с суммированием чисел. Ниже приведен хороший пример: естественное (но намного более сложное) обобщение задачи планирования.

Допустим, дано множество из n задач, которые должны выполняться на одном компьютере. Для каждого задания i установлено время доступности $r_i$, когда оно впервые становится доступным для обработки; предельное время $d_i$, к которому оно должно быть завершено; и продолжительность обработки $t_i$. Будем считать, что все эти параметры являются целыми числами. Чтобы задание i было завершено, ему должен быть выделен смежный набор из $t_i$ единиц времени где-то в интервале $[r_i, d_i]$. В любой момент на машине может выполняться только одно задание. Вопрос заключается в следующем: можно ли спланировать все задания для выполнения так, чтобы каждое задание было завершено к предельному времени? Назовем этот экземпляр задачей планирования с временем доступности и предельным временем.

***Утверждение 27.23.*** Задача планирования с временем доступности и предельным временем является NP-полной.

# Co-NP и асимметрия NP

Чтобы расширить свое представление об этом общем классе задач, вернемся к тем определениям, на которых базировался класс NP. Мы уже видели, что понятие эффективного сертифицирующего алгоритма не предполагает конкретного алгоритма для фактического решения задачи, который бы превосходил метод «грубой силы».

***А теперь еще одно наблюдение:*** определение эффективного сертифицирования (а следовательно, и NP) по своей сути асимметрично. Входная строка s представляет экземпляр с положительной сертификацией в том, и только в том случае, если существует короткое значение t, для которого B(s, t) = да. Из отрицания этого утверждения мы видим, что входная строка представляет экземпляр с отрицательной сертификацией в том, и только в том случае, если для всех коротких t выполняется B(s, t) = нет.

Все это близко к нашим интуитивным представлениям о NP: если экземпляр сертифицируется положительно, мы можем привести короткое доказательство этого факта. Но для экземпляров с отрицательной сертификацией определение не гарантирует соответствующего короткого доказательства; ответ отрицателен просто потому, что не удается найти строку, которая бы служила доказательством. Если набор условий невыполним, какие «доказательства» могли бы быстро убедить вас в невыполнимости задачи?

# Co-NP и асимметрия NP

Для каждой задачи X существует естественная дополняющая задача $\bar{X}$: для всех входных строк s мы говорим, что $s \in X$ в том, и только в том случае, если $s \notin \bar{X}$. Следует заметить, что если $X \in P$, то $\bar{X} \in P$, так как из алгоритма A, решающего X, мы можем просто построить алгоритм $\bar{A}$, который выполняет A, а затем «инвертирует» его ответ.

Но при этом совершенно не очевидно, что из $X \in NP$ должно следовать, что $\bar{X} \in NP$. Вместо этого задача $\bar{X}$ обладает другим свойством: для всех s выполняется $s \in \bar{X}$ в том, и только в том случае, если для всех t длины не более $p(|s|)$, B(s, t) = нет. Это принципиально иное определение, которое невозможно обойти простым «инвертированием» вывода эффективного сертифицирующего алгоритма B для получения $\bar{B}$. Проблема в том, что «существует t» в определении NP превращается в «для всех t», и это серьезное изменение.

Существует класс задач, параллельных NP, спроектированных для моделирования этой проблемы; ему присвоено достаточно естественное название co-NP. Задача X принадлежит co-NP в том, и только в том случае, если дополняющая задача $\bar{X}$ принадлежит NP.

# Co-NP и асимметрия NP

Но мы не знаем наверняка, что NP и co-NP различны; можем только спросить:

***Утверждение 27.24.*** Выполняется ли NP = co-NP? И снова принято считать, что NP ≠ co-NP: из того, что экземпляры задач с положительной сертификацией имеют короткие доказательства, совершенно не следует, что экземпляры с отрицательной сертификацией также должны иметь короткие доказательства.

Доказательство NP ≠ co-NP стало бы еще более важным шагом, чем доказательство P ≠ NP, по следующей причине:

***Утверждение 27.25.*** Если NP ≠ co-NP, то P ≠ NP.

Доказательство. На самом деле мы докажем обратное утверждение: из P = NP следует NP = co-NP. Здесь важно то, что множество P замкнуто относительно дополнения; следовательно, если P = NP, то множество NP также будет замкнуто относительно дополнения. Или в более формальном изложении, начиная с предположения P = NP, имеем

$$X \in NP \Longrightarrow X \in P \Longrightarrow \bar{X} \in P \Longrightarrow \bar{X} \in NP \Longrightarrow X \in co-NP$$

и

$$X \in co-NP \Longrightarrow \bar{X} \in NP \Longrightarrow \bar{X} \in P \Longrightarrow X \in P \Longrightarrow X \in NP$$

Из этого следует, что $NP \subseteq co-NP$ и $co-NP \subseteq NP$, а значит, NP = co-NP. ■

# Хорошая характеризация: класс NP ∩ co-NP

Если задача X принадлежит как NP, так и co-NP, то она обладает следующим полезным свойством: для ответа «да» существует короткое доказательство; для ответа «нет» также существует короткое доказательство. Задачи, принадлежащие этому пересечению NP ∩ co-NP, называются имеющими хорошую характеризацию, так как для решения всегда существует хороший сертификат.

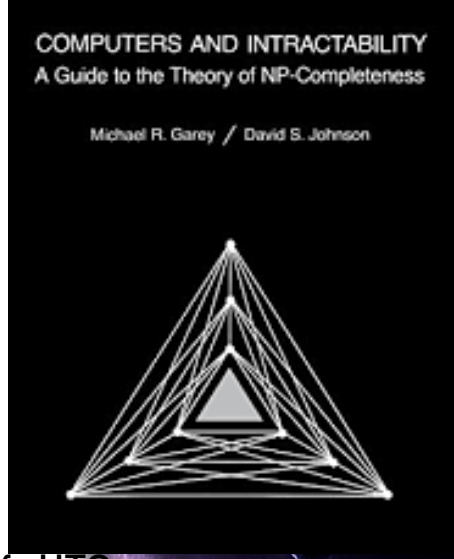
Это понятие напрямую соответствует некоторым результатам, которые мы видели ранее. Например, возьмем задачу определения того, содержит ли потоковая сеть поток с величиной как минимум v, для некоторого значения v. Чтобы доказать, что ответ на этот вопрос положительный, достаточно продемонстрировать поток, достигающий этой величины; это свойство согласуется с принадлежностью задачи NP. Но также можно доказать и отрицательный ответ: можно продемонстрировать разрез, пропускная способность которого строго меньше v. Этот дуализм между экземплярами с положительной и отрицательной сертификацией является сутью теоремы о максимальном потоке и минимальном разрезе.

Если задача X находится в P, то она принадлежит как NP, так и co-NP; следовательно, P ⊆ NP ∩ co-NP.

этот вопрос также остается открытым:

***Утверждение 27.26.*** Выполняется ли P = NP ∩ co-NP?

# Другие NP-трудные задачи




Буквально для тысяч задач была доказана их NP-трудность. Большинство этих доказательств вы можете найти в классической «Черной страшной книге по NP-полноте» (Scary Black Book of NP-Completeness) Гэри(Garey) и Джонсона (Johnson)
Garey M. R. et al. A Guide to the Theory of NP-Completeness //Computers and intractability. – 1990. – С. 37-79.

- **PlanarCircuitSat:** задана логическая схема, которая может быть встроена в плоскость так, что никакие два провода не пересекаются. Существуют ли входные данные, которые формируют вывод схемы True? NP-трудность этой задачи можно доказать редукцией от общей задачи выполнимости схемы, заменив каждое пересечение проводов небольшой комбинацией вентилей.
- **1-in-3Sat:** задана формула 3CNF. Существует ли присваивание значений переменным такое, что каждая компонента содержит ровно один литерал True? NP-трудность этой задачи можно доказать редукцией от обычной задачи 3Sat.
- **NotAllEqual3Sat:** задана формула 3CNF. Существует ли присваивание значений переменным такое, что каждая компонента содержит как минимум один литерал True и как минимум один литерал False? NP-трудность этой задачи можно доказать редукцией от обычной задачи 3Sat.
- **Planar3Sat:** задана формула 3CNF. Рассмотреть двудольный граф, вершинами которого являются компоненты и переменные, а ребра обозначают, что переменная (или ее отрицание) присутствует в компоненте. Если этот граф плоский, то формула 3CNF также называется плоской. Задача Planar3Sat требует при заданной формуле 3CNF определить, имеется ли для этой формулы выполнимое присваивание. NP-трудность этой задачи можно доказать редукцией от задачи PlanarCircuitSat

# Другие NP-трудные задачи

- **Exact3DimensionalMatching** или **X3M**: задано множество S и группа подмножеств S из трех элементов, называемых тройками. Существует ли подгруппа непересекающихся троек, которая полностью покрывает S? NP-трудность этой задачи можно доказать редукцией от за-дачи 3Sat, поскольку в ней имеются три элемента.
- **SetCover:** задана группа множеств S = {S1, S2, …, Sn]. Найти наименьшую подгруппу множеств S, содержащую все элементы U.S. Эта задача является обобщением двух задач: VertexCover и X3M.
- **HittingSet**: задана группа множеств S = {S1, S2,…, Sm}. Найти наименьшее число элементов в $\bigcup_i S_i$, которые попадают в каждое множеств S. Эта задача также является обобщением задачи VertexCover.

В дополнение к этим сухим, но полезным задачам было показано, наиболее интересные головоломки и игры-пасьянсы типа «солитер» таже являются NP-трудными или имеют NP-трудные обобщения. Вот несколько примеров: Minesweeper (Сапер) (от CircuitSat) (ссылка 1 и 2); Sudoku (Судоку) (по существу, от 3Sat); Tetris (Тетрис) (от 3Partition); Klondike, он же Solitaire (Солитер) (от 3Sat); Pac-Man (Пакман) (от HamiltonianCycle); Super Mario Brothers (Братья Супер-Марио) (от 3Sat) ссылка 2; Candy Crush Saga (от варианта 3Sat); Threes/2048 (от 3Sat, разумеется); Trainyard (Ж/д стрелки) (от DominatingSet;); кратчайшее решение кубика Рубика n×n×n (от 3Sat через особый случай задачи PlanarUndirectedHamCycle); Cookie Clicker (от Partition Или 3Partition).

# Что дальше? PSPACE

P и NP - это всего лишь первые два шага в огромной иерархии классов сложности. Полиномиальное пространство PSPACE - это множество задач на принятие решений, которые мог быть решены с использованием полиномиального пространства (polyncmial space). Каждая задача в NP (следовательно, и в P) также находится NP-трудность PSPACE. В общем, все верят, что NP ≠ PSPACE, но никто не может доказать даже, что P ≠ PSPACE.

***Определение 27.6.*** Задача ∏ является PSPACE-трудной, если для любой задачи ∏', которая может быть решена с использованием полиномиального пространства, существует множественная редукция, выполняемая за полиномиальное время, от ∏' к П.

Если любая PSPACE-трудная задача находится в NP, то PSPACE = NP. Аналогично если любая PSPACE-трудная задача находится в P, то PSPACE = P.

Канонической PSPACE-трудной является задача квантифицированной логической формулы (quantified boolean formula - QBF): если задана логическая формула Ф, которая может включать любое число кванторов все-общности или существования, но без свободных переменных, то равна ли формула Ф значению True? Например, следующее выражение представляет собой корректные входные данные для задачи QBF: $\exists a\colon \forall b\colon \exists c\colon (\forall d\colon a \vee b \vee c \vee \bar{d}) \Leftrightarrow ((b \wedge c) \vee (\exists e\colon (\overline{\bar{a} \Longrightarrow e}) \vee (c \neq a \wedge e)))$.

Другая каноническая PSPACE-трудная задача - NFA totality: если задан недетерминированный конечный автомат (НКА) M на некотором алфавите ∑, то принимает ли M каждую строку в ∑*?

# Экспоненциальное время

Следующий значительно больший класс сложности EXP (также называемый EXPTIME) - это множество задач на принятие решений, которые могут быть решены за экспоненциальное время, т. е. с использованием не более 2n шагов при некоторой константе c > 0. Каждая задача в PSPACE(следовательно, в NP (следовательно, в P)) также находится в EXP. В общем, все верят, что PSPACE ⊊ EXP, но никто не может доказать даже, что NP ≠ EXP. Задача П является EXP-трудной, если для любой задачи П', которую можно решить за экспоненциальное время, существует множественная редукция, выполняемая за полиномиальное время, от П' к П. Если любая EXP-трудная задача находится в PSPACE, то EXP = PSPACE. Аналогично если любая EXP-трудная задача находится в NP, то EXP = NP. Мы точно знаем, что P ≠ EXP, в частности, ни одна из EXP-трудных задач не находится в P.

Естественные обобщения многих интересных игр с двумя участниками, таких как шашки, шахматы, калах и го, действительно являются EXP-трудными задачами. Граница между PSPACE-полными и EXP-трудными играми весьма тонка.

# Все выше и выше!

***Определение 27.7.*** NEXP - это класс задач на принятие решений, которые можно решить за недетерминированное экспоненциальное время. Равнозначно: задача на принятие решений находится в NEXP, если и только если для каждого варианта Yes существует доказательство этого факта, которое можно проверить за экспоненциальное время.

***Определение 27.8.*** EXPSPACE - это множество задач на принятие решений, которые можно решить, используя экспоненциальное пространство.

Даже эти классы большей сложности содержат трудные задачи; например, если мы добавим оператор пересечения в синтаксис регулярных выражений, то определение, описывают ли два таких выражения один и тот же язык - EXPSPACE-трудная задача.

Все классы сложности можно упорядочить по вложенности:

$$P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP \subseteq EXPSPACE \subseteq EEXP \subseteq NEEXP \subseteq \cdots$$

Пределом этой последовательности классов монотонно возрастающей сложности является класс ELEMENTARY задач на принятие решений, которые могут быть решены с использованием времени или пространства,ограниченного функцией формы $2 \uparrow^k$ для некоторого постоянного целого числа k, где

$$2 \uparrow^k := \begin{cases} n, \text{если } k = 0, \\ 2^{2\uparrow^{k-1} n} \text{ иначе.} \end{cases}$$

Например, $2 \uparrow^1 n = 2^n$ и $2 \uparrow^2 n = 2^{2^n}$. $\forall$ естественно решаемая задача может быть решена за элементарное время? Ларри Стокмайер 1974 г – ДКА не может (ссылка).